

Специальные разделы математической статистики

Введение в математическую статистику, предельные теоремы теории вероятностей

Урок 1. Введение

- Математическая статистика - важная дисциплина в Data Science.
- Она помогает описывать, анализировать, прогнозировать и проверять гипотезы.
- Навыки математической статистики необходимы для принятия решений на основе данных.
- Для успешного освоения курса требуется понимание случайной величины, законов распределения и предельных теорем.
- В первом модуле курса будут повторены и углублены знания из математического анализа.
- Основные понятия теории вероятностей будут рассмотрены, включая генеральную совокупность и выборку.
- Будут изучены основные параметры для описания данных и базовые принципы работы с данными.
- Рекомендуется пройти тест для проверки знаний и настроиться на изучение курса.

Урок 2. Базовые понятия теории вероятностей

В тексте подробно объясняются основные понятия теории вероятностей:

1. Вероятностное пространство — это тройка $(\Omega, \mathcal{A}, P)$, где:
 - Ω — множество всех возможных исходов эксперимента;
 - $\mathcal{A}$ — σ -алгебра подмножеств Ω , представляющих события;
 - P — вероятностная мера, ставящая в соответствие каждому событию из $\mathcal{A}$ вероятность его наступления.
2. Случайная величина — это функция $\xi : \Omega \rightarrow \mathbb{R}$, которая каждому исходу $\omega \in \Omega$ ставит в соответствие числовое значение $\xi(\omega)$.
3. Дискретная случайная величина — принимает конечное или счётное множество значений, для каждого из которых вероятность строго больше нуля.
4. Непрерывная случайная величина — описывается плотностью распределения $p_\xi(x)$, и вероятность попадания в любую область B вычисляется через интеграл:
$$P(\xi \in B) = \int_B p_\xi(x) dx.$$
5. Функция распределения случайной величины $F_\xi(x)$ — вероятность того, что случайная величина ξ примет значение, меньшее x :
 - $F_\xi(x) = P(\xi < x)$.

6. Математическое ожидание $M\xi$ — среднее значение случайной величины, взвешенное по её вероятностям:
1. Для дискретных случайных величин: $M\xi = \sum_k \xi_k P(\xi = \xi_k)$;
 2. Для непрерывных случайных величин: $M\xi = \int_{-\infty}^{\infty} x p\xi(x) dx$.
7. Дисперсия $D\xi$ — мера рассеяния случайной величины вокруг её математического ожидания:
- $D\xi = M((\xi - M\xi)^2) = M(\xi^2) - (M\xi)^2$.

Эти понятия являются фундаментальными в теории вероятностей и широко используются в различных областях, включая статистику, машинное обучение и анализ данных.

- Теория вероятностей - фундаментальная часть математической статистики.
- Основные понятия теории вероятностей: случайная величина, дискретные и непрерывные распределения, функция распределения, математическое ожидание и дисперсия.
- Случайная величина - функция $\xi : \Omega \rightarrow \mathbb{R}$, множество значений которой принадлежит σ -алгебре.
- Дискретная случайная величина - принимает только конечное или счетное множество значений.
- Непрерывная случайная величина - имеет непрерывное распределение, плотность которого описывает распределение вероятности.
- Функция распределения случайной величины - $F_\xi(x) = P(\xi < x)$, монотонна, непрерывна слева, имеет пределы на бесконечности.
- Математическое ожидание - $M\xi = \int_{-\infty}^{\infty} x dF_\xi(x)$, линейность, неотрицательность, аддитивность.
- Дисперсия - $D\xi = M((\xi - M\xi)^2)$, мера рассеяния случайной величины вокруг её математического ожидания.

Урок 3. Основные распределения случайных величин

Равномерное непрерывное распределение описывает ситуацию, когда случайная величина может принимать любое значение в определённом интервале **(a, b)**, и каждое из этих значений имеет одинаковую вероятность. Это как если бы вы бросили мяч на отрезок и он с равной вероятностью мог остановиться в любой точке этого отрезка.

Показательное (экспоненциальное) распределение используется для моделирования времени между событиями, которые происходят независимо друг от друга и с постоянной интенсивностью. Например, оно может описывать время между звонками в колл-центр или время между отказами оборудования. Вероятность того, что событие произойдёт в определённый момент времени, уменьшается экспоненциально с течением времени.

Нормальное распределение (Гаусса) — это один из самых важных законов в статистике и теории вероятностей. Оно описывает распределение значений вокруг среднего значения, причём большинство значений группируется вокруг среднего, а экстремальные значения встречаются редко. Это распределение часто встречается в природе и используется для моделирования различных явлений, таких как рост людей, результаты тестов и т. д.

- Дискретное равномерное распределение: случайная величина ξ принимает конечное множество значений $a_1, a_2, \dots, a_n$ с одинаковой вероятностью.
- Показательное (экспоненциальное) распределение: случайная величина ξ имеет плотность вероятности $p\xi(x)$, задаётся формулой: $p\xi(x) = \lambda e^{-\lambda x}$, $x \geq 0$, 0 , $x < 0$.
- Функция распределения $F\xi(x)$ определяется как: $F\xi(x) = P(\xi \leq x)$.

- Нормальное распределение: случайная величина ξ имеет плотность вероятности $p_{\xi}(x)$, задаётся формулой: $p_{\xi}(x) = 1/\sqrt{(2\pi\sigma^2)} \exp(- (x - \mu)^2 / 2\sigma^2)$.
- Стандартная нормальная случайная величина: θ , если она имеет нормальное распределение с параметрами $\mu=0$ и $\sigma = 1$.
- Правило “трех сигм”: практически достоверным является тот факт, что нормально распределённая случайная величина ξ примет значение из промежутка $(\mu - 3\sigma, \mu + 3\sigma)$.
- FWHM (полная ширина на половине максимума): это характеристика формы кривой или функции, которая показывает ширину графика между точками, где значение функции равно половине её максимального значения.

Урок 4. Пределные теоремы теории вероятностей

Закон больших чисел (ЗБЧ)

Закон больших чисел — это принцип в теории вероятностей, который гласит, что среднее значение большого количества независимых случайных величин с одинаковым распределением будет близко к их математическому ожиданию.

ПРИМЕР ДЛЯ ОДИНАКОВО РАСПРЕДЕЛЁННЫХ СЛУЧАЙНЫХ ВЕЛИЧИН:

Если у нас есть последовательность случайных величин $\xi_1, \xi_2, \xi_3, \dots, \xi_n$, которые независимы и имеют одинаковое распределение, и если у них есть конечный второй момент (то есть $M(\xi_i^2) < \infty$ для всех i), то среднее значение этих величин будет стремиться к их математическому ожиданию $M\xi$ по вероятности:

$$\frac{1}{n} \sum_{i=1}^n \xi_i \rightarrow M\xi \text{ при } n \rightarrow \infty.$$

Это означает, что чем больше случайных величин мы усредняем, тем ближе среднее значение будет к математическому ожиданию.

Центральная предельная теорема (ЦПТ)

Центральная предельная теорема — это ещё один важный принцип в теории вероятностей. Она гласит, что сумма большого количества независимых и одинаково распределённых случайных величин будет иметь распределение, близкое к нормальному (гауссову) распределению, даже если исходные случайные величины не имеют нормального распределения.

ФОРМУЛИРОВКА ЦПТ:

Пусть $\xi_1, \xi_2, \xi_3, \dots, \xi_n$ — последовательность независимых и одинаково распределённых случайных величин с конечной дисперсией ($0 < D\xi < \infty$). Обозначим $M\xi = a$ — математическое ожидание каждой случайной величины и $D\xi = \sigma^2$ — дисперсию каждой случайной величины. Рассмотрим сумму этих случайных величин:

$$S_n = \xi_1 + \xi_2 + \xi_3 + \dots + \xi_n.$$

Тогда нормированная сумма $\frac{S_n - nM\xi}{\sigma\sqrt{n}}$ будет стремиться к стандартному нормальному распределению $N(0, 1)$ при $n \rightarrow \infty$.

СЛЕДСТВИЕ ЦПТ

Из ЦПТ следует, что для любых вещественных чисел $x < y$:

$$P(x < \frac{S_n - nM\xi}{\sigma\sqrt{n}} < y) \rightarrow \Phi(y) - \Phi(x),$$

где $\Phi(x)$ — функция распределения стандартного нормального закона. Это означает, что вероятность попадания нормированной суммы в интервал $[x, y]$ приближается к вероятности попадания стандартной нормальной случайной величины в тот же интервал.

Предельная теорема Муавра — Лапласа

Теорема Муавра — Лапласа является частным случаем центральной предельной теоремы и применяется для схемы Бернулли. Она утверждает, что если $\mathbf{v}_n(\mathbf{A})$ — число успехов в n независимых испытаниях с вероятностью успеха p , то нормированная величина:

$$\frac{\nu_n(A) - np}{\sqrt{np(1-p)}} \rightarrow N(0,1), \text{ где } N(0,1) \text{ — стандартное нормальное распределение. Это}$$

означает, что при больших n распределение числа успехов $\mathbf{v}_n(\mathbf{A})$ можно аппроксимировать нормальным распределением с параметрами:

- Математическое ожидание: $M\nu_n(A) = np$.
- Дисперсия: $D\nu_n(A) = np(1-p) = npq$.
- Предельные теоремы теории вероятностей играют ключевую роль в математической статистике и ее приложениях.
- Закон больших чисел позволяет оценить приближение среднего значения выборки к истинному математическому ожиданию.
- Центральная предельная теорема объясняет, почему явления в природе и обществе можно моделировать с помощью нормального распределения.
- Теорема Муавра-Лапласа позволяет аппроксимировать биномиальное распределение нормальным, упрощая вычисления в практических задачах.
- Без понимания предельных теорем невозможно эффективно анализировать и интерпретировать статистические данные.
- Сходимость по вероятности описывает приближение последовательности случайных величин к предельному значению.
- Центральная предельная теорема объясняет, почему нормальное распределение часто встречается в природе и данных.
- Предельная теорема Муавра-Лапласа применяется для схемы Бернулли и позволяет аппроксимировать распределение числа успехов нормальным распределением.

Урок 5. Основные понятия математической статистики

Основная идея текста: чтобы изучить свойства некоторой случайной величины (например, время загрузки веб-страницы), мы используем выборку — набор значений, полученных в результате нескольких наблюдений. На основе этой выборки можно сделать выводы о свойствах всей совокупности возможных значений случайной величины.

Термины и объяснения:

1. Генеральная совокупность — это все возможные значения, которые может принимать случайная величина. Например, если мы измеряем рост людей, то генеральная совокупность — это рост всех людей в мире.
2. Выборка — это часть генеральной совокупности, набор значений, полученных в результате наблюдений. Например, если мы измерили рост 100 человек, то эти 100 значений образуют выборку.

3. Эмпирическое распределение — это распределение, построенное на основе выборки. Оно позволяет оценить свойства неизвестного распределения генеральной совокупности.
4. Выборочные характеристики — это показатели, которые описывают свойства выборки. К ним относятся:
 - Выборочное среднее — среднее арифметическое значений в выборке.
 - Выборочная дисперсия — показывает разброс значений в выборке относительно выборочного среднего.
 - Выборочные моменты — используются для оценки характеристик распределения или проверки статистических гипотез.
5. Гистограмма — это график, который показывает распределение значений в выборке. Она строится по сгруппированным данным и позволяет визуальнo оценить форму распределения.

Пример:

Предположим, мы хотим изучить время загрузки веб-страницы. Мы проводим 100 измерений и получаем выборку из 100 значений времени загрузки. На основе этой выборки мы можем вычислить среднее время загрузки и его дисперсию, а также построить гистограмму, которая покажет, как распределены значения времени загрузки.

- Генеральная совокупность - множество возможных значений случайной величины.
- Выборкой называется часть генеральной совокупности, доступная для наблюдения.
- Цель математической статистики - сделать выводы о свойствах неизвестного распределения на основе выборки.
- Эмпирическое распределение - распределение, построенное на основе выборки.
- Выборочные характеристики включают выборочное среднее, выборочную дисперсию, выборочный центральный k-ый момент.
- Гистограмма - другой характеристикой распределения является закон распределения для дискретных распределений или плотность для абсолютно непрерывных.
- Число интервалов группировки выбирается по формуле Стерджесса.

Урок 6. Точечные оценки параметров распределения

Основные понятия математической статистики

ОЦЕНКА ПАРАМЕТРОВ

Оценка параметра — это статистика, которая используется для приближения неизвестного параметра распределения на основе выборки данных. Например, если мы хотим оценить среднее значение генеральной совокупности, мы можем использовать среднее значение выборки.

ТИПЫ ОЦЕНОК

- **Несмещённая оценка:** оценка называется несмещённой, если её математическое ожидание равно истинному значению оцениваемого параметра. Это означает, что в среднем оценка не даёт систематических ошибок.
- **Состоятельная оценка:** оценка считается состоятельной, если она сходится по вероятности к истинному значению параметра при увеличении объёма выборки. Это означает, что чем больше данных, тем точнее оценка.

СВОЙСТВА ГИСТОГРАММЫ

Гистограмма — это способ визуализации распределения данных, который делит диапазон значений на интервалы и показывает количество данных в каждом интервале.

Теорема о свойствах гистограммы: площадь столбца гистограммы над интервалом с ростом объёма выборки сходится по вероятности к площади под графиком истинной плотности над этим же интервалом. Это означает, что гистограмма становится более точной приближением истинного распределения при увеличении объёма данных.

ВЫБОРОЧНЫЕ МОМЕНТЫ

Выборочные моменты — это характеристики данных, которые помогают описать их распределение. Например, выборочное среднее показывает центральную тенденцию данных, а выборочная дисперсия отражает разброс данных вокруг среднего значения.

- **Выборочное среднее:** является несмещённой и состоятельной оценкой для теоретического среднего (математического ожидания).
- **Выборочные моменты k-го порядка:** являются несмещёнными, состоятельными и асимптотически нормальными оценками для теоретических k-го момента.

ВЫБОРОЧНАЯ ДИСПЕРСИЯ

Выборочная дисперсия определяется как мера разброса данных вокруг выборочного среднего.

- **Теоретическая дисперсия:** выборочная дисперсия является состоятельной оценкой для теоретической дисперсии, но не является несмещённой. Для устранения смещения используется исправленная выборочная дисперсия.

ИСПРАВЛЕННАЯ ВЫБОРОЧНАЯ ДИСПЕРСИЯ:

$$[S_0^2 = \frac{n}{n-1} S^2]$$

где (S^2) — выборочная дисперсия. Исправленная выборочная дисперсия является несмещённой и состоятельной оценкой для теоретической дисперсии.

- Основная задача аналитика - делать выводы о данных, находить средние значения, оценивать разброс и определять вероятности.
- Точечные оценки - способ "угадать" неизвестные параметры на основе данных.
- Статистика - функция от эмпирических данных, предназначена для оценивания неизвестного параметра.
- Оценка - случайная величина, свойства оценок: несмещенность и состоятельность.
- Несмещенность: математическое ожидание оценки равно истинному значению параметра.
- Состоятельность: оценка сходится по вероятности к истинному значению параметра при увеличении объёма выборки.
- Гистограмма - один из самых простых и наглядных способов визуализации распределения данных.
- Выборочные моменты - ключевые характеристики данных, описывают их распределение.
- Выборочное среднее - несмещенная и состоятельная оценка для математического ожидания.
- Выборочная дисперсия - состоятельная оценка для дисперсии, но не несмещенная.
- Исправленная выборочная дисперсия - несмещенная и состоятельная оценка для дисперсии.

Урок 7. Метод моментов получения точечных оценок

Основная идея текста: **метод моментов** — это способ оценить параметры распределения данных (например, среднюю цену автомобилей) путём сравнения выборочных характеристик (среднего, дисперсии) с их теоретическими аналогами.

Простыми словами:

- **Теоретический момент** — это математическое ожидание какой-либо функции от данных, например, среднее значение или второй момент (связанный с дисперсией).
- **Выборочный момент** — это та же характеристика, но вычисленная на основе имеющихся данных.

Мы подбираем параметр θ так, чтобы теоретический и выборочный моменты совпали.

Пример:

Предположим, у нас есть данные о ценах на автомобили, и мы хотим оценить среднюю цену (θ). Мы можем использовать выборочное среднее значение цен как оценку для θ .

СОСТОЯТЕЛЬНОСТЬ ОЦЕНКИ:

Оценка метода моментов является состоятельной, что означает, что при увеличении объёма выборки оценка будет приближаться к истинному значению параметра.

Важно:

Хотя оценка метода моментов часто является состоятельной, она не всегда является несмещённой (то есть её математическое ожидание не всегда равно истинному значению параметра).

Пример с равномерным распределением:

Если данные распределены равномерно на отрезке $[0; \theta]$, то метод моментов позволяет найти оценку для θ , сравнивая выборочное среднее с теоретическим средним.

- Метод моментов используется для оценки параметров распределения данных.
- Метод основан на сравнении выборочных и теоретических моментов.
- Метод прост в использовании и часто применяется в задачах машинного обучения.
- Оценка метода моментов выбирается так, чтобы теоретический момент совпадал с выборочным.
- Функция $g(y)$ выбирается так, чтобы существовал момент $Mg(\xi) = h(\theta)$.
- Оценка метода моментов определяется как $\hat{\theta} = h^{-1}(1/n \sum i = 1^n g(X_i))$.
- Оценка метода моментов является состоятельной, но не всегда несмещённой.
- Пример показывает, что оценка метода моментов может быть смещённой.

Урок 8. Метод максимального правдоподобия получения точечных оценок

Метод максимального правдоподобия — это способ определить такие значения параметров распределения, при которых вероятность получить наблюдаемые данные становится максимально возможной.

Представьте, что вы бросаете игральную кость и хотите узнать, честно ли она сделана. Если кость честная, то вероятность выпадения каждого числа от 1 до 6 должна быть одинаковой. Метод максимального правдоподобия поможет вам определить, какие значения параметров (в данном случае вероятности выпадения каждого числа) наиболее соответствуют вашим наблюдениям.

Функция правдоподобия — это математическое выражение, которое показывает, насколько вероятно получить наблюдаемые данные при определённых значениях параметров. Она рассчитывается как произведение вероятностей для каждого наблюдения.

Например, если у вас есть выборка из нескольких бросков честной кости, функция правдоподобия будет учитывать вероятности выпадения каждого числа в этой выборке.

Оценка максимального правдоподобия (ОМП) — это значение параметра, при котором функция правдоподобия достигает максимума. То есть это такое значение параметра, которое делает наблюдаемые данные наиболее вероятными.

В примере с распределением Пуассона, если у нас есть выборка данных, мы можем использовать метод максимального правдоподобия, чтобы найти наиболее подходящее значение параметра λ , которое наилучшим образом объясняет наши данные. В результате вычислений мы выясняем, что ОМП параметра λ равна выборочному среднему значению наших данных.

- Метод максимального правдоподобия позволяет найти значения параметров, максимизирующие вероятность получения наблюдаемых данных.
- Широко используется в логистической регрессии, классификации, байесовских моделях и естественных науках.
- Функция правдоподобия определяется произведением плотностей распределения или вероятностей.
- Оценкой максимального правдоподобия (ОМП) является значение параметра, при котором достигается максимум функции правдоподобия.
- Логарифмическая функция правдоподобия может быть упрощена, превращая произведение в сумму.
- Пример: нахождение ОМП для распределения Пуассона, где плотность распределения и функция правдоподобия определены.
- Вторая производная функции правдоподобия показывает, что точка максимума находится в точке выборочного среднего.

Вероятностные распределения. Построение графиков распределения

Урок 1. Введение

В первом модуле мы изучили основы теории вероятностей и статистики, познакомились с ключевыми понятиями, такими как случайные величины, их распределения, математическое ожидание и дисперсия. Эти знания стали фундаментом для понимания того, как данные могут быть описаны и анализированы. Все полученные знания в первом модуле закреплялись практическими задачами, которые в основном решались аналитически, без использования программных инструментов. Во втором модуле мы сосредоточимся на применении этих концепций с использованием современных программных средств.

В этом модуле мы углубимся в практическое применение концепций математической статистики. Мы будем изучать распределения вероятностей — один из важнейших инструментов анализа данных, который позволяет не только описывать наблюдаемые закономерности, но и делать прогнозы на основе исторических данных.

Распределения вероятностей помогают нам ответить на вопросы:

- Какова вероятность наступления события?
- Какие значения наиболее вероятны?
- Как можно визуализировать и интерпретировать данные?

Представьте, что вам нужно решить задачу прогноза некоторого события, как правило, выражающегося в численном значении. Это может быть предсказание количества посетителей сайта, количества очков в спортивной стрельбе, цены на акции и т. д. При этом есть три существенных момента:

- у вас есть некоторый набор исторических данных этого события, на основании которого вы будете делать прогноз;
- важно не просто предсказать результат, но и указать степень его достоверности;
- нужно также выделить основные особенности набора данных, например: интервал разброса значений, среднее значение, медиана, мода и т. д.

Математически эта задача формулируется так: «На основе анализа имеющихся данных определить вероятность будущего события той же природы, что и данные, и выделить основные закономерности в данных».

Задача значительно усложняется, если объем данных настолько велик, что их невозможно обработать вручную. Соответственно, возникает вопрос: как подходить к поставленной задаче?

Пример

Возьмём простой пример с подбрасыванием монеты. В случае, если 5 раз подряд выпадет «решка», для многих людей кажется очевидным, что в шестой раз выпадет «орёл», хотя вероятность по-прежнему $1/2$. Это так называемая «ошибка игрока». Ещё многим кажется,

что комбинация из 6 «решек» менее вероятна, чем комбинация «5 орлов и 1 решка», хотя на самом деле вероятность обеих комбинаций одинакова при условии, что за решкой закреплено конкретное место в ряду (например, последний шестой бросок).

Эти примеры показывают, что человеческая интуиция часто вводит нас в заблуждение при анализе вероятностей. Поэтому в данном модуле мы будем опираться на строгий математический аппарат, чтобы избежать подобных ошибок.

Определение Данный модуль посвящён такому методу обработки данных, как теория распределения вероятностей — разделу математической статистики о законах, описывающих область значений случайной величины и соответствующие вероятности появления этих значений.

Основные темы модуля:

- Введение в теорию распределения вероятностей. Здесь будут представлены основные математические определения и термины, необходимые для решения поставленной задачи.
- Библиотека SciPy языка программирования Python. Это специальная библиотека, разработанная для быстрой работы с разными типами распределений.
- Библиотеки Matplotlib и Seaborn языка программирования Python. Данные библиотеки — хороший инструмент визуализации данных.
- Дискретные распределения вероятностей. В данной теме будут описаны распределения, используемые для описания дискретных (счётных, отдельных, прерывистых) значений.
- Абсолютно непрерывные распределения вероятностей. Здесь речь пойдёт о распределениях для непрерывных величин, которые изменяются плавно, «без скачков».

! Важно: Этот модуль научит вас использовать язык программирования Python для анализа данных, что является важным навыком для ML-разработчиков, аналитиков и инженеров данных. Перед тем как приступить к практическим задачам, давайте введем основные математические термины и определения из статистики.

Урок 2. Терминология распределения вероятностей

Итак, мы говорили о распределении вероятности реализации того или иного значения из некоторого набора данных. В этом уроке мы “освежим” основные понятия теории вероятностей из первого модуля, которые являются фундаментальной частью математической статистики. Эти понятия помогают нам описывать случайные явления, анализировать данные и строить модели машинного обучения.

Пример

Представьте, что вы анализируете данные о количестве покупок в интернет-магазине или прогнозируете количество клиентов в салоне красоты. Чтобы делать точные выводы и прогнозы, важно понимать, как распределены данные. Например, если большинство клиентов приходит в салон в определённое время дня, это может помочь оптимизировать расписание работы сотрудников. В этом уроке мы познакомимся с ключевыми терминами и концепциями, которые помогут вам работать с такими задачами.

Определение Распределение вероятностей — это математический закон, который описывает, насколько вероятно каждое возможное значение случайной величины. Ещё говорят, что это математическое описание подмножеств выборочного пространства. Выборочное пространство Ω — это набор всех возможных исходов наблюдаемого случайного явления. Например:

- подбрасывание монеты $\Omega = \{\text{орел}\}, \{\text{решка}\}$;
- измерение температуры воздуха $\Omega = [-40C^{\circ}; 40C^{\circ}]$
- анализ результатов теста (выборочное пространство может быть множеством всех возможных баллов) $\Omega = \{0, 1, 2, \dots, 100\}$.

Основные термины:

Определение

- Случайная величина — математическая формализация, зависящая от случайных событий. Например, количество очков при броске игральной кости.
- Событие — результат эксперимента, характеризующийся своей вероятностью. Например, выпадение "орла" при подбрасывании монеты.
- Вероятность — относительная мера наступления события. Например, вероятность выпадения "орла" равна 0.5.
- Математическое ожидание — средневзвешенное значение всех возможных значений случайной величины с учётом их вероятностей. Например, если бросить кубик много раз, среднее значение выпавших чисел будет равно 3.5.
- Медиана — значение, которое делит упорядоченный ряд пополам. Например, если выписать все значения по возрастианию, медиана будет находиться ровно посередине.
- Мода — значение с наибольшей вероятностью. Например, если чаще всего встречается число 4, то это мода.
- Дисперсия — мера разброса значений вокруг среднего. Чем больше разброс, тем выше дисперсия.

Классы распределений:

Определение Дискретные распределения: значения фиксированные, например, количество орлов при броске монеты.

Непрерывные распределения: значения могут быть любыми в заданном диапазоне, например, температура воздуха.

Функция вероятности и функция распределения:

Определение Функция вероятности — описывает вероятность наступления события. Функция распределения — показывает вероятность того, что случайная величина примет значение меньше или равное определённому числу.

Распределение вероятностей, пространство выборки которого является одномерным (например, действительные числа, список меток, упорядоченные метки или двоичное), называется одномерным, в то время как распределение, пространство выборки которого представляет собой векторное пространство размерности 2 или более, называется многомерным.

Одномерное распределение даёт вероятности того, что одна случайная величина примет различные значения; многомерное распределение (совместное распределение вероятностей) даёт вероятности того, что случайный вектор — список из двух или более случайных величин — примет различные комбинации значений.

Распределение вероятностей может быть описано в различных формах, что зависит от класса. Одно из самых общих описаний, которое применяется для абсолютно непрерывных и дискретных переменных, — с помощью функции вероятности $P: A \rightarrow R$, где A — входное пространство событий, R — множество действительных чисел из отрезка $[0;1]$, равных вероятности наступления события.

Чаще изучаются распределения вероятностей, аргументом которых являются подмножества наборов чисел, то есть A — набор чисел. В таком случае принято обозначать $P(X \in E)$ как вероятность того, что определённое значение переменной X принадлежит определённому событию E .

В частном случае действительной случайной величины распределение вероятностей может быть эквивалентно представлено кумулятивной функцией распределения. Кумулятивная функция распределения случайной величины X относительно распределения вероятностей P определяется как:

$$F(x) = P(X \leq x)$$

То есть получилась функция, оценивающая вероятность того, что X для случайной величины будет принято значение, меньшее или равное x .

Кумулятивная функция распределения (CDF) любой вещественной случайной величины обладает свойствами:

Определение $F(x)$ — неубывающая функция.

$F(x)$ — непрерывная функция.

$$\lim_{x \rightarrow -\infty} F(x) = 0, \quad \lim_{x \rightarrow +\infty} F(x) = 1.$$

$$P(a < X \leq b) = F(b) - F(a).$$

! Важно: Мы разобрали на этом уроке основные классы распределений вероятностей и терминологию, необходимую для дальнейшей работы с ними. В следующем уроке мы научимся применять эти знания на практике с помощью программных инструментов, таких как SciPy, Matplotlib и Seaborn. Это позволит нам не только понимать распределения, но и визуализировать их. Перед началом следующего юнита, предлагаем вам немного попрактиковаться, освежить и закрепить полученные знания.

Урок 3. Программные инструменты

Итак, наша основная задача — анализ данных через распределение вероятностей. Для этого используются программные библиотеки Python, такие как SciPy, Matplotlib и Seaborn.

Эти инструменты позволяют эффективно работать с данными, строить графики и проводить статистический анализ. В этом уроке мы рассмотрим основные возможности этих библиотек и их применение для анализа распределений.

Библиотека SciPy

Определение SciPy — это библиотека Python, предназначенная для решения научных и инженерных задач. Она включает модуль stats, который предоставляет инструменты для работы с распределениями вероятностей.

Модуль `scipy.stats`

- содержит большое количество распределений (нормальное, экспоненциальное, биномиальное и др.);
- позволяет вычислять плотности вероятности, функции распределения, генерировать случайные величины и проводить статистические тесты;

Пример кода, который строит график плотности вероятности стандартного нормального распределения $N(0,1)$:

```
from scipy.stats import norm          # для работы с нормальным
распределением
import numpy as np                    # для создания массива чисел
import matplotlib.pyplot as plt       # для построения графиков

x = np.linspace(-4, 4, 100)          # Генерирует 100 точек в
диапазоне от -4 до 4.
pdf = norm.pdf(x, loc=0, scale=1)     # вычисление плотности вероятности
plt.plot(x, pdf, label="N(0, 1)")     # рисует кривую
plt.legend()                          # добавляет легенду
plt.show()                           # отображает график
```

Библиотека Matplotlib

С данной библиотекой вы наверняка уже знакомы из курса Python для анализа данных.

Определение Библиотека Matplotlib — библиотека языка Python, предназначенная для визуализации данных через двумерную и трёхмерную графику.

Matplotlib поддерживает множество графиков. Выделим основные:

- Линейные графики — для показа изменения одной или нескольких переменных во времени или по другому непрерывному параметру. Можно использовать разные цвета, стили и маркеры линий для различения переменных. Например, можно построить несколько линейных графиков для сравнения температуры воздуха в разных городах в течение года.
- Столбчатые графики — для показа сравнения одной или нескольких переменных по разным категориям или группам. Можно использовать вертикальные или горизонтальные столбцы, а также столбцы разной ширины и высоты для различения переменных. Например, можно построить несколько столбчатых графиков для показа продаж разных продуктов в разных регионах.

- Круговые диаграммы — для показа долей или процентов одной переменной по разным категориям или группам. Можно использовать разные цвета, размеры и углы секторов для различения категорий. Например, можно построить несколько круговых графиков для показа распределения населения по возрасту, полу и доходу.
- Точечные графики — для показа взаимосвязи или корреляции между двумя или несколькими переменными. Можно использовать разные цвета, формы и размеры точек для различения переменных. Например, можно построить несколько точечных графиков для изучения зависимости роста растений от уровня освещения, воды и удобрений.
- Гистограммы — для показа распределения одной или нескольких переменных по интервалам значений. Можно использовать разные цвета, ширину и высоту столбцов для различения переменных. Например, можно построить несколько гистограмм для анализа частоты встречаемости разных слов в тексте.

Кроме указанных, возможны также: контурные графики, ломаные линии, кривые Безье, поля векторов, поверхности и т. д.

Ссылка на документацию библиотеки: [Matplotlib](#).

Пример кода для построения гистограммы с использованием Matplotlib:

```
import matplotlib.pyplot as plt      # импортируем библиотеку
data = [1, 2, 2, 3, 4, 4, 4, 5]     # определяем массив данных
plt.hist(data, bins=5, alpha=0.7, color='blue', edgecolor='black') #
# рисуем гистограмму с 5 бинами, прозрачность столбцов 0.7, цвет заливки
# синий и границ – черный
plt.title("Гистограмма распределения") # название графика

#подписи осей
plt.xlabel("Значения")
plt.ylabel("Частота")
plt.show()
```

Библиотека Seaborn

Определение Seaborn — это библиотека для создания статистической графики на Python. Она основана на Matplotlib и тесно интегрируется со структурами данных Pandas.

Функции построения графиков в Seaborn работают с фреймами данных и массивами, содержащими целые наборы данных, и внутренне выполняют необходимое семантическое отображение и статистическую агрегацию для получения информативных графиков.

Основное отличие от Matplotlib в том, что Seaborn требует меньшего написания кода при создании графиков. Однако Seaborn более ограничен и не имеет такой широкой коллекции графиков, как Matplotlib, поскольку ориентирован на конкретные методы статистического анализа.

Подробнее про Seaborn по [ссылке](#).

Пример кода для построения гистограммы с использованием Seaborn:

```
import seaborn as sns # импорт библиотек
import pandas as pd

# создаем датафрейм из 100 случайных чисел, взятых из стандартного
# нормального распределения
```

```
data = pd.DataFrame({'x': np.random.normal(size=100)})
```

```
# построение гистограммы с KDE (Kernel Density Estimation) – гладкую кривую плотности.  
sns.histplot(data['x'], kde=True, color='green')  
plt.title("Распределение с плотностью")  
plt.show()
```

Сейчас мы не будем глубоко погружаться во все возможности библиотек, поскольку в этом нет необходимости. В рамках данного урока мы ввели необходимые нам библиотеки и указали ссылки на их документацию, к которой можно обратиться для изучения их возможностей. Далее на конкретных примерах распределений будет продемонстрировано, как обращаться к описанию распределений через модуль stats библиотеки SciPy и как визуализировать их с помощью Matplotlib и Seaborn. Однако, для удовлетворения собственного любопытства, вы можете попробовать попрактиковаться и решить следующие задачи.

Задачи

Задача 3.1

Постройте график плотности вероятности нормального распределения $N(0,1)$ на интервале $[-5;5]$ с использованием модуля stats библиотеки SciPy.

- заполнение области под кривой цветом с прозрачностью 50%;
- вертикальные линии на $\pm 1\sigma$ (стандартное отклонение) пунктиром;
- подписи осей и заголовков в LaTeX-формате: $N(0,1)$;
- Seaborn-стиль сетки (`sns.set_theme()`).

Задача 3.2

Сгенерируйте выборку из 2000 значений из экспоненциального распределения с параметром $\lambda=1.5$. Постройте:

- гистограмму с автоматическим подбором бинов (`bins='auto'`);
- теоретическую плотность распределения (используя `scipy.stats.expon.pdf`);
- вертикальную линию на математическом ожидании $1/\lambda$ красным цветом;
- легенду и подпись оси X: "Время до события (секунды)".

Задача 3.3

Для биномиального распределения с $n=15$, $p=0.3$:

- рассчитайте вероятности $P(k)$ для всех k от 0 до n (`scipy.stats.binom.pmf`);
- постройте столбчатую диаграмму (`.bar()`), где каждый столбец соответствует $P(k)$;
- добавьте маркеры точек поверх столбцов (`.bar() marker='o', color='red'`);
- Подпишите ось X: "Число успехов", ось Y: "Вероятность".

Задача 3.4

Создайте DataFrame с данными:

- категории: ["A", "B", "C", "D"];
- доли: [25, 40, 20, 15] (в процентах);

Постройте круговую диаграмму с:

- выносками для каждой категории (`autopct='%1f%%'`);
- тенью (`shadow=True`);
- отделением сектора "B" на 0.1 (`shadow=Trueexplode=[0, 0.1, 0, 0]`);
- seaborn-стилем (`sns.set_style("whitegrid")`).

Задача 3.5

Сгенерируйте данные:

- $x \sim U[0, 10]$
 - равномерное распределение в интервале $[0, 10]$ (100 точек);
- $y = 3x + \epsilon$, где $\epsilon \sim N(0, 4)$;

постройте точечный график (`sns.scatterplot`), добавив:

- линию регрессии `sns.scatterplots.regplot` с `sns.scatterplot`), кривую, которая аппроксимирует зависимость между переменными x и y методом наименьших квадратов без отображения доверительного интервала вокруг линии;
- подписи осей в формате LaTeX: $y = 3x + N(0, 4)$;
- ограничение по оси Y: $[\min(y) - 2, \max(y) + 2]$;
- точки с прозрачностью 0.7.

[Документация модуля stats библиотеки SciPy.](#)

[Документация библиотеки Matplotlib.](#)

[Документация библиотеки Seaborn.](#)

Урок 4. Дискретные распределения вероятностей

В предыдущих уроках мы обсуждали основы распределений вероятностей и их роль в анализе данных. В этом уроке мы сосредоточимся на дискретных распределениях — одном из ключевых инструментов для моделирования данных, которые принимают конечное или счётное множество значений. Дискретные распределения широко применяются в задачах машинного обучения, прогнозирования и анализа данных, например, для прогнозирования числа мошеннических транзакций в день или числа кликов на рекомендуемый товар.

Определение Дискретное распределение вероятностей — это распределение, которое описывает вероятности событий для случайной величины, принимающей только счётное множество значений. Вероятность любого события E можно выразить как сумму вероятностей отдельных исходов:

$$P(X \in E) = \sum_{\omega \in A \cap E} P(X = \omega).$$

где X — случайная величина, ω — число из счётного множества A исходов событий E , причём $P(X \in A) = 1$. Для дискретного распределения ключевым понятием является массовая функция вероятности (PMF), которая показывает вероятность того, что случайная величина примет конкретное значение: $P(X = \omega)$.

Описывается дискретное распределение вероятностью наступления каждого из возможных исходов события. Как и для любого распределения для дискретных событий определены понятия мат. ожидания и дисперсии. Однако следует понимать, что мат. ожидание для дискретного случайного события — величина в общем случае нереализуемая как исход единичного случайного события, а скорее величина, к которой будет стремиться среднее арифметическое исходов событий при увеличении их количества.

Дискретное равномерное распределение

Дискретное равномерное распределение — это симметричное распределение вероятностей, при котором все возможные значения случайной величины наблюдаются с одинаковой вероятностью. Если случайная величина может принимать n различных значений, то вероятность каждого из них равна: $P(X = x_i) = \frac{1}{n}$, $i = 1, 2, \dots, n$.

Пример

Простым примером дискретного равномерного распределения является бросание честного кубика. Возможные значения: 1, 2, 3, 4, 5, 6 и вероятность каждого исхода равна:

$$P(X = x_i) = \frac{1}{6}, k \in \{1, 2, 3, 4, 5, 6\}.$$

Если бросить две игральные кости и сложить их значения, результирующее распределение перестанет быть равномерным. Например, сумма 7 будет встречаться чаще, чем 2 или 12, так как существует больше комбинаций, дающих 7.

Хотя дискретные равномерные распределения часто описываются на множестве целых чисел, они могут быть определены на любом конечном множестве значений. Например, можно рассмотреть случайную величину, принимающую значения $\{a, b, c\}$ с вероятностями:

$$3P(X = a) = P(X = b) = P(X = c) = \frac{1}{3}$$

Для наглядности рассмотрим графики функции массы вероятности (PMF) и функции распределения (CDF) дискретного равномерного распределения.

Определение Функция массы вероятности показывает вероятность каждого значения случайной величины. Для дискретного равномерного распределения все возможные значения имеют одинаковую вероятность: $P(X = k) = \frac{1}{n}$, где n — количество возможных значений. Для броска кубика график PMF будет выглядеть как шесть столбиков одинаковой высоты, соответствующих значениям 1,2,3,4,5,6.

Рассмотрим пример построения графиков PMF с использованием языка программирования Python

```
import numpy as np                                # Импортируем библиотеку
numpy                                              # Импортируем метод randint
from scipy.stats import randint                  # Импортируем библиотеку
import matplotlib.pyplot as plt                  # Импортируем библиотеку
matplotliblib

# Создаем поле для графика
plt.figure(figsize=(15, 10))

# Определяем диапазоны значений
x1 = np.arange(1, 5)                             # Значения для n = 4
x2 = np.arange(1, 8)                             # Значения для n = 7
x3 = np.arange(1, 16)                            # Значения для n = 15

# Строим графики PMF
plt.scatter(x1, randint.pmf(x1, 1, 5), s=60, c="blue", marker="o",
label='n = 4')
plt.scatter(x2, randint.pmf(x2, 1, 8), s=80, c="red", marker="*",
label='n = 7')
plt.scatter(x3, randint.pmf(x3, 1, 16), s=120, c="green", marker="+",
label='n = 15')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='upper right')

# Отображаем график
plt.title("PMF дискретного равномерного распределения")
plt.xlabel("Значения")
plt.ylabel("Вероятность")
plt.show()
```

На полученном графике синие точки ($n=4$), возможные значения: 1,2,3,4. Соответственно, вероятность каждого значения равна $1/4=0.25$.

Красные звездочки ($n=7$), возможные значения: 1,2,3,4,5,6,7. Вероятность каждого значения: $\frac{1}{7} \approx 0.143$.

Зеленые крестики ($n=15$), возможные значения: 1,2,...,15. Вероятность каждого значения: $\frac{1}{15} \approx 0.067$.

! Важно: для всех значений, которые не входят в заданный диапазон, вероятность равна 0. Это означает, что на графике точки вне диапазона просто отсутствуют. Определение Функция распределения (CDF) показывает вероятность того, что случайная величина примет значение, меньшее или равное заданному. Для дискретного

равномерного распределения график CDF будет ступенчатым, где каждая "ступенька" соответствует накопленной вероятности.

Пример

Если случайная величина принимает значения {1,2,3,4}, то: вероятность каждого значения:

$P(X = k) = \frac{1}{4}$; накопленная вероятность (CDF): $P(X \leq 1) = 0.25$; $P(X \leq 2) = 0.5$; $P(X \leq 3) = 0.75$;

$P(X \leq 4) = 1$; График CDF для броска честного кубика будет состоять из шести "ступенек", где каждая ступенька соответствует накопленной вероятности значений 1,2,3,4,5,6.

Функция CDF для дискретного равномерного распределения реализуется на Python следующим образом

```
import numpy as np                                # Импортируем библиотеку
numpy                                              # Импортируем метод randint
from scipy.stats import randint                  # Импортируем библиотеку
import matplotlib.pyplot as plt                  # Импортируем библиотеку
matplotlib

# Создаем поле для графика
plt.figure(figsize=(15, 10))

# Определяем диапазоны значений
x = np.arange(0, 25, 0.001)                    # Создаем массив чисел от 0
до 25 с шагом 0.001

# Строим графики CDF
plt.plot(x, randint.cdf(x, 1, 5), c="blue", label='n = 4') # CDF для n
= 4
plt.plot(x, randint.cdf(x, 1, 8), c="red", label='n = 7')  # CDF для n
= 7
plt.plot(x, randint.cdf(x, 1, 16), c="green", label='n = 15') # CDF для
n = 15

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='lower right')

# Отображаем график
plt.title("CDF дискретного равномерного распределения")
plt.xlabel("Значения")
plt.ylabel("Накопленная вероятность")
plt.show()
```

! Важно: Важно отметить, что на каждой «ступеньке» графиков правая точка является выколотой. Как читать такой график? «Если в наборе 4 значения, то вероятность того, что случайная величина примет значение меньше или равное второму, равна $\frac{1}{2}$ » или «Если в наборе 15 значений, то вероятность того, что случайная величина примет значение меньше или равное восьмому, равна $\frac{8}{15}$ ».

Основные параметры дискретного равномерного распределения:

Параметр	Значение
Обозначение	$U\{\}$

Параметры	$a, b \in \mathbb{Z} (b > a)$
Размер диапазона	$n = b - a + 1$
Выборка — возможные случаи	$k \in \{a, a+1, \dots, b-1, b\}$
PMF	$\{n\} \frac{1}{n}$
CDF	$\{n\} \frac{\lfloor k \rfloor - a + 1}{n}$
Среднее	$\frac{a + b}{2}$
Мода	-
Дисперсия	$\frac{n^2 - 1}{12}$

Подробнее о методах для равномерного дискретного распределения можно найти [здесь](#).

Распределение Бернулли

Определение Пусть исход события может принимать одно из двух значений — 0 или 1 с вероятностями p и q соответственно, — тогда распределение вероятности получения каждого из предложенных исходов будет называться распределением Бернулли.

Фактически, его можно рассматривать как модель для набора возможных результатов любого отдельного эксперимента, в котором задаётся вопрос «да-нет». Такие вопросы приводят к результатам с логическим значением: один бит, значение которого равно «успеху» с вероятностью p и «неудаче» с вероятностью q . Его можно использовать для представления подбрасывания монеты, где 1 и 0 будут представлять «орёл» и «решка» соответственно, а p будет вероятностью падения монеты стороной «орёл» или наоборот. PMF показывает вероятности каждого из двух возможных значений (0 и 1):

$$P(X = k) = \begin{cases} q = 1 - p, & k = 0 \\ p, & k = 1 \end{cases}$$

Посмотрим, как графически показать PMF и CDF распределения Бернулли в Python.

```
import numpy as np          # Импортируем библиотеку numpy
from scipy.stats import bernoulli # Импортируем метод bernoulli
import matplotlib.pyplot as plt # Импортируем библиотеку matplotlib

# Создаем поле для графика
plt.figure(figsize=(15, 10))

# Определяем диапазон значений
x = np.arange(0, 2)        # Значения 0 и 1

# Строим графики PMF
plt.scatter(x, bernoulli.pmf(x, 0.3), s=60, c="blue", marker="o", label='p = 0.3')
plt.scatter(x, bernoulli.pmf(x, 0.6), s=80, c="red", marker="*", label='p = 0.6')
plt.scatter(x, bernoulli.pmf(x, 0.8), s=120, c="green", marker="x", label='p = 0.8')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='upper center')

# Отображаем график
plt.title("PMF распределения Бернулли")
plt.xlabel("Значения")
```

```
plt.ylabel("Вероятность")
plt.show()
```

Точки соответствуют вероятностям $P(X=0)=q$ и $P(X=1)=p$. Например, при $p=0.3$, $P(X=0)=0.7$, $P(X=1)=0.3$.

Функция CDF показывает накопленную вероятность для значений k :

$$F(X \leq k) = \begin{cases} 0, & k < 0 \\ 1 - p, & 0 \leq k < 1 \\ 1, & k \geq 1 \end{cases}$$

```
import numpy as np                # Импортируем библиотеку numpy
from scipy.stats import bernoulli # Импортируем метод bernoulli
import matplotlib.pyplot as plt   # Импортируем библиотеку matplotlib
```

```
# Создаем поле для графика
plt.figure(figsize=(15, 10))
```

```
# Определяем диапазон значений
x = np.arange(-0.5, 1.5, 0.001)
```

```
# Строим графики CDF
plt.plot(x, bernoulli.cdf(x, 0.3), c="blue", marker="o", label='p = 0.3')
plt.plot(x, bernoulli.cdf(x, 0.6), c="red", marker="*", label='p = 0.6')
plt.plot(x, bernoulli.cdf(x, 0.8), c="green", marker="x", label='p = 0.8')
```

```
# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='lower center')
```

```
# Отображаем график
plt.title("CDF распределения Бернулли")
plt.xlabel("Значения")
plt.ylabel("Накопленная вероятность")
plt.show()
```

На графике наблюдается “всплеск” в точках $k=0$ и $k=1$. Например, при $p=0.3$: $F(X<0)=0$; $F(X\leq 0)=0.7$; $F(X\leq 1)=1$.

Основные параметры дискретного равномерного распределения:

Параметр	Значение
Обозначение	$Bin_{p,q}(x)$
Параметры	$0 \leq p \leq 1$, $q=1-p$
Выборка — возможные случаи	$k \in \{0, 1\}$
PMF	p , $k=1$
CDF	0 , $k < 0$ $1-p$, $0 \leq k < 1$ 1 , $k \geq 1$
Среднее	p

Мода	$0, p < \frac{1}{2}$ $[0,1], p = \frac{1}{2}$ $1, p > \frac{1}{2}$
Дисперсия	pq

Подробнее о методах для распределения Бернулли можно найти [здесь](#).

Распределение Бернулли является частным случаем биномиального распределения, при котором проводится одно испытание.

Биномиальное распределение

Определение Биномиальное распределение — это дискретное распределение вероятностей количества успешных результатов в последовательности из n независимых экспериментов, в каждом из которых задаётся вопрос «да-нет», и у каждого свой логический результат: успех (с вероятностью p) или неудача (с вероятностью $1-p=q$).

Одиночный эксперимент с успехом или неудачей также называется испытанием Бернулли или экспериментом Бернулли, а последовательность результатов называется процессом Бернулли; для одного испытания биномиальное распределение является распределением Бернулли.

Биномиальное распределение часто используется для моделирования количества успехов в выборке размером n , полученной из совокупности размером N таким образом, что любой элемент / результат эксперимента может реализовываться несколько раз (такая выборка называется выборкой с заменой).

Пример

Пример — вытаскивание шаров из корзины с последующим возвращением. Если выборка выполняется без замены, розыгрыши не являются независимыми, и поэтому результирующее распределение является гипергеометрическим распределением, а не биномиальным.

Определение PMF биномиального распределения — это вероятность получения последовательности из n испытаний Бернулли, в которой первые k испытаний являются «успешными», а остальные (последние) испытания приводят к «неудаче». Поскольку испытания независимы, а вероятности между ними остаются постоянными, любая последовательность (перестановка) имеет одинаковую вероятность достижения (независимо от положения успехов в последовательности). Отсюда возникает

биномиальный коэффициент, равный числу перестановок.
$$P(X = k) = \frac{n!}{k!(n-k)!} p^k \cdot q^{n-k}$$

Если величины x и y имеют биномиальные распределения с параметрами $\{n_x; p\}$ и $\{n_y; p\}$ соответственно, то их сумма также будет распределена биномиально с параметрами $\{n_x + n_y; p\}$.

Посмотрим, как графически показать PMF и CDF биномиального распределения в Python.

```
import numpy as np
from scipy.stats import binom
import matplotlib.pyplot as plt

# Создаем поле для графика
plt.figure(figsize=(15, 10))
```

```

# Определяем параметры
n, p = 20, 0.5
x = np.arange(0, n + 1)
plt.scatter(x, binom.pmf(x, n, p), c='b', label='n=20, p=0.5')

n, p = 40, 0.5
x = np.arange(0, n + 1)
plt.scatter(x, binom.pmf(x, n, p), c='r', label='n=40, p=0.5')

n, p = 20, 0.7
x = np.arange(0, n + 1)
plt.scatter(x, binom.pmf(x, n, p), c='g', label='n=20, p=0.7')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='upper left')

# Отображаем график
plt.title("PMF биномиального распределения")
plt.xlabel("Количество успехов (k)")
plt.ylabel("Вероятность")
plt.show()

```

На графике синие точки (n=20,p=0.5) - симметричное распределение. Красные точки (n=40,p=0.5) для более широкого диапазон значений. Зеленые точки (n=20,p=0.7) - асимметричное распределение (смещено вправо).

Функция CDF показывает накопленную вероятность для значений

$$F(X \leq k) = \sum_{i=0}^k P(X = i)$$

```

import numpy as np
from scipy.stats import binom
import matplotlib.pyplot as plt

# Создаем поле для графика
plt.figure(figsize=(15, 10))

# Определяем параметры
n, p = 20, 0.5
x = np.arange(0, n + 5, 0.01)
plt.plot(x, binom.cdf(x, n, p), c='b', label='n=20, p=0.5')

n, p = 40, 0.5
x = np.arange(0, n + 5, 0.01)
plt.plot(x, binom.cdf(x, n, p), c='r', label='n=40, p=0.5')

n, p = 20, 0.7
x = np.arange(0, n + 5, 0.01)
plt.plot(x, binom.cdf(x, n, p), c='g', label='n=20, p=0.7')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='upper left')

```

```
# Отображаем график
plt.title("CDF биномиального распределения")
plt.xlabel("Количество успехов (k)")
plt.ylabel("Накопленная вероятность")
plt.show()
```

Основные параметры биномиального распределения:

Параметр	Значение
Обозначение	$Bin_{p,q}(k)$
Параметры	$n \in \{0, 1, 2, \dots\}$ $0 \leq p \leq 1, q = 1 - p$
Выборка - количество успехов	$k \in \{0, 1, \dots, n\}$
PMF	$\frac{n!}{k!(n-k)!} p^k q^{n-k}$
CDF	Регуляризованная неполная бета-функция
Среднее	np
Мода	$\lfloor (n+1)p \rfloor$
Дисперсия	npq

Подробнее о методах для биномиального распределения можно найти [здесь](#).

Геометрическое распределение

Определение Геометрическое распределение — это дискретное распределение вероятностей, которое описывает количество испытаний k , необходимых для получения первого успеха в последовательности независимых испытаний Бернулли. Каждое испытание имеет два возможных исхода: успех с вероятностью p и неудача с вероятностью $q = 1 - p$. В теории вероятностей и статистике геометрическое распределение представляет собой одно из двух дискретных распределений вероятностей: распределение вероятностей числа X испытаний Бернулли, необходимых для получения одного успеха, выполняется на множестве $N = \{0, 1, 2, \dots\}$; распределение вероятностей числа $Y = X - 1$ неудач до первого успеха, выполняется на множестве $\mathbb{N}_0 = \{1, 2, 3, \dots\}$.

Эти два разных геометрических распределения не следует путать друг с другом. Часто первое называют сдвинутое геометрическое распределение; однако, чтобы избежать двусмысленности, разумно явно указать, что подразумевается, указав выборку.

Посмотрим, как графически показать PMF и CDF геометрического распределения в Python. Функция PMF геометрического распределения показывает вероятность того, что первый успех произойдет на k -м испытании: $P(X = k) = (1 - p)^{k-1} p$

```
import numpy as np
from scipy.stats import geom
import matplotlib.pyplot as plt
```

```
# Создаем поле для графика
plt.figure(figsize=(15, 10))
```

```
# Определяем параметры
p = 0.2
```



```

x = np.arange(1, 11) # Значения от 1 до 10
plt.scatter(x, geom.pmf(x, p), c='b', label='p=0.2')

p = 0.5
plt.scatter(x, geom.pmf(x, p), c='r', label='p=0.5')

p = 0.7
plt.scatter(x, geom.pmf(x, p), c='g', label='p=0.7')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='upper right')

# Отображаем график
plt.title("PMF геометрического распределения")
plt.xlabel("Количество испытаний до первого успеха (k)")
plt.ylabel("Вероятность")
plt.show()

```

На графике синие точки ($p=0.2$) - распределение более "растянуто", так как вероятность успеха мала. Красные точки ($p=0.5$), распределение более сбалансировано. Зеленые точки ($p=0.7$) распределение "сжато", так как вероятность успеха высока. Функция CDF показывает накопленную вероятность для значений k :

$$F(X \leq k) = 1 - (1 - p)^{[k]}, k \geq 1.$$

```

import numpy as np
from scipy.stats import geom
import matplotlib.pyplot as plt

# Создаем поле для графика
plt.figure(figsize=(15, 10))

# Определяем параметры
p = 0.2
x = np.arange(1, 10, 0.01)
plt.plot(x, geom.cdf(x, p), c='b', label='p=0.2')

p = 0.5
plt.plot(x, geom.cdf(x, p), c='r', label='p=0.5')

p = 0.7
plt.plot(x, geom.cdf(x, p), c='g', label='p=0.7')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='upper right')

# Отображаем график
plt.title("CDF геометрического распределения")
plt.xlabel("Количество испытаний до первого успеха (k)")
plt.ylabel("Накопленная вероятность")
plt.show()

```

Линии имеют "всплеск" в точках k , соответствующих целым значениям. Например, при $p=0.2$, $F(X \leq 5) \approx 0.67$ или $F(X \leq 10) \approx 0.89$.

Основные параметры геометрического распределения:

Параметр	Значение
Обозначение	$Geom_p$
Параметры	$0 \leq p \leq 1$
Выборка - количество испытаний до успеха	$k \in \mathbb{N}$
PMF	$(1 - p)^{k-1} p$
CDF	$0, x < 1$ $1 - (1 - p)^{\lfloor x \rfloor}, x \geq 1$
Среднее	$\frac{1}{p}$
Мода	1
Медиана	$\left\lceil \frac{-1}{\log_2(1 - p)} \right\rceil$
Дисперсия	$\frac{1 - p}{p^2}$

Подробнее о методах для геометрического распределения можно найти [здесь](#).

Геометрическое распределение также является частным случаем отрицательного биномиального распределения.

Отрицательное биномиальное распределение

Определение Отрицательное биномиальное распределение — это дискретное распределение вероятностей, которое моделирует количество неудач в последовательности независимых и идентично распределённых испытаний Бернулли до того, как произойдёт заданное (неслучайное) количество успехов (обозначается r).

Пример

Подбрасывание кубика: сколько раз выпадет не 6 (неудача) до третьего выпадения 6 (успех)? Тестирование оборудования: сколько дней оборудование будет работать (неудача) до третьего сбоя (успех)?

В таком случае распределение вероятностей количества возникающих сбоев будет отрицательным биномиальным распределением.

Альтернативной формулировкой является моделирование количества общих испытаний (вместо количества неудач). Фактически, для заданного (неслучайного) числа успехов r количество неудач $n - r$ являются случайными, поскольку общее количество испытаний n являются случайными.

Например, мы могли бы использовать отрицательное биномиальное распределение для моделирования количества дней n (случайных), в течение которых работает определённая машина (указанная через r), прежде чем она выйдет из строя.

Распределение Паскаля и распределение Поля являются частными случаями отрицательного биномиального распределения. Среди инженеров, климатологов и других специалистов принято использовать «Паскаля» для случая целочисленного параметра

времени остановки r и использовать «Поля» для вещественного случая. Чаще используется «Паскаль».

При $r=1$, мы получим геометрическое распределение для величины $k+1$.

Посмотрим, как графически показать PMF и CDF отрицательного биномиального распределения в Python.

Функция PMF отрицательного биномиального распределения показывает вероятность получения k неудач до r -го успеха: $P(X = k) = \binom{k+r-1}{k} \cdot p^r \cdot q^k$, где $\binom{k+r-1}{k}$ - биномиальный коэффициент,

```
import numpy as np
from scipy.stats import nbinom
import matplotlib.pyplot as plt

# Создаем поле для графика
plt.figure(figsize=(15, 10))

# Определяем параметры
n, p = 2, 0.1
x = np.arange(0, n * 30, 1)
plt.scatter(x, nbinom.pmf(x, n, p), label='r=2, p=0.1')

n, p = 40, 0.5
x = np.arange(0, n * 1.5, 1)
plt.scatter(x, nbinom.pmf(x, n, p), label='r=40, p=0.5')

n, p = 15, 0.4
x = np.arange(0, n * 4, 1)
plt.scatter(x, nbinom.pmf(x, n, p), label='r=15, p=0.4')

n, p = 2, 0.3
x = np.arange(0, n * 30, 1)
plt.scatter(x, nbinom.pmf(x, n, p), label='r=2, p=0.3')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='upper right')

# Отображаем график
plt.title("PMF отрицательного биномиального распределения")
plt.xlabel("Количество неудач (k)")
plt.ylabel("Вероятность")
plt.show()
```

На графике синие точки ($r=2, p=0.1$) - распределение смещено вправо из-за малой вероятности успеха. Красные точки ($r=40, p=0.5$) - более сбалансированное распределение. Зеленые и другие цвета: различные комбинации r и p .

Функция CDF показывает накопленную вероятность для значений k :

$F(X \leq k) = I_p(r, k + 1)$, где $I_p(r, k + 1)$ - регуляризованная неполная бета-функция.

```
import numpy as np
from scipy.stats import nbinom
```

```

import matplotlib.pyplot as plt

# Создаем поле для графика
plt.figure(figsize=(15, 10))

# Определяем параметры
n, p = 2, 0.1
x = np.arange(0, n * 30, 0.01)
plt.plot(x, nbinom.cdf(x, n, p), label='r=2, p=0.1')

n, p = 40, 0.5
x = np.arange(0, n * 1.5, 0.01)
plt.plot(x, nbinom.cdf(x, n, p), label='r=40, p=0.5')

n, p = 15, 0.4
x = np.arange(0, n * 4, 0.01)
plt.plot(x, nbinom.cdf(x, n, p), label='r=15, p=0.4')

n, p = 2, 0.3
x = np.arange(0, n * 30, 0.01)
plt.plot(x, nbinom.cdf(x, n, p), label='r=2, p=0.3')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='lower right')

# Отображаем график
plt.title("CDF отрицательного биномиального распределения")
plt.xlabel("Количество неудач (k)")
plt.ylabel("Накопленная вероятность")
plt.show()

```

Основные параметры отрицательного биномиального распределения:

Параметр	Значение
Обозначение	$NB_{r,p}(k)$
Параметры	$0 \leq p \leq 1, r > 0$
Выборка — количество неудач	$k \in \mathbb{N}$
PMF	$\frac{(k + r - 1)!}{k!(r - 1)!} p^r q^k$
CDF	Упорядоченная неполная бета-функция
Среднее	$\frac{r(1 - p)}{p}$
Мода	$\left\lfloor \frac{(r - 1)(1 - p)}{p} \right\rfloor, r > 1$
Дисперсия	$\frac{r(1 - p)}{p^2}$

Подробнее о методах для отрицательного биномиального распределения можно найти [здесь](#).

Гипергеометрическое распределение

Определение Пусть общая совокупность содержит N объектов, из них D помечены как «1», а $N-D$ как «0». Будем считать выбор объекта с меткой «1», как успех, а с меткой «0» как неудачу. Проведём n испытаний, причём выбранные объекты больше не будут участвовать в дальнейших испытаниях (без возвращения). Вероятность наступления k успехов будет подчиняться гипергеометрическому распределению.

Таким образом, мы перешли к ситуации без возвращения и описываем вероятность количества успешных выборов из совокупности с заранее известным количеством успехов и неудач (заранее известное количество белых и чёрных мячей в корзине, козырных карт в колоде, бракованных деталей в партии и т. д.). Причём вероятность исхода меняется от испытания к испытанию. При больших N и малых n/N гипергеометрическое распределение можно аппроксимировать биномиальным распределением.

Посмотрим, как графически показать PMF и CDF гипергеометрического распределения в Python. Функция PMF гипергеометрического распределения показывает вероятность

получения ровно k успехов в выборке:
$$P(X = k) = \frac{\binom{D}{k} \binom{N-D}{n-k}}{\binom{N}{n}},$$
 где:

- $\binom{a}{b}$ — число сочетаний,
- N — размер совокупности,
- D — количество успешных объектов в совокупности,
- n — размер выборки,
- k — количество успехов в выборке ($k \in [\max(0, n + D - N), \min(n, D)]$).

```
from scipy.stats import hypergeom
import matplotlib.pyplot as plt
import numpy as np
```

```
# Создаем поле для графика
plt.figure(figsize=(15, 10))
```

```
# Определяем параметры
[M, n, N] = [200, 70, 80]
rv = hypergeom(M, n, N)
x = np.arange(0, n + 1)
plt.scatter(x, rv.pmf(x), label='N=200, D=70, n=80')
```

```
[M, n, N] = [200, 70, 160]
rv = hypergeom(M, n, N)
x = np.arange(0, n + 1)
plt.scatter(x, rv.pmf(x), label='N=200, D=70, n=160')
```

```
[M, n, N] = [200, 40, 80]
rv = hypergeom(M, n, N)
x = np.arange(0, n + 1)
plt.scatter(x, rv.pmf(x), label='N=200, D=40, n=80')
```

```
[M, n, N] = [100, 40, 80]
rv = hypergeom(M, n, N)
```

```

x = np.arange(0, n + 1)
plt.scatter(x, rv.pmf(x), label='N=100, D=40, n=80')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='upper right')

# Отображаем график
plt.title("PMF гипергеометрического распределения")
plt.xlabel("Количество успехов (k)")
plt.ylabel("Вероятность")
plt.show()

```

Функция CDF показывает накопленную вероятность для значений k:

$$F(X \leq k) = \sum_{i=\max(0, n+D-N)}^k P(X = i)$$

```

from scipy.stats import hypergeom
import matplotlib.pyplot as plt
import numpy as np

# Создаем поле для графика
plt.figure(figsize=(15, 10))

# Определяем параметры
[M, n, N] = [200, 70, 80]
rv = hypergeom(M, n, N)
x = np.arange(0, n + 1)
plt.scatter(x, rv.cdf(x), label='N=200, D=70, n=80')

[M, n, N] = [200, 70, 160]
rv = hypergeom(M, n, N)
x = np.arange(0, n + 1)
plt.scatter(x, rv.cdf(x), label='N=200, D=70, n=160')

[M, n, N] = [200, 40, 80]
rv = hypergeom(M, n, N)
x = np.arange(0, n + 1)
plt.scatter(x, rv.cdf(x), label='N=200, D=40, n=80')

[M, n, N] = [100, 40, 80]
rv = hypergeom(M, n, N)
x = np.arange(0, n + 1)
plt.scatter(x, rv.cdf(x), label='N=100, D=40, n=80')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='lower right')

# Отображаем график
plt.title("CDF гипергеометрического распределения")
plt.xlabel("Количество успехов (k)")
plt.ylabel("Накопленная вероятность")
plt.show()

```

Основные параметры гипергеометрического распределения:

Параметр	Значение
Обозначение	$HG_{N,D,n}(k)$
Параметры	$N=\{0,1,2,\dots\}$ $D=\{0,1,2,\dots,N\}$ $n=\{0,1,2,\dots,N\}$
Выборка — количество успехов	$k \in \{\max(0, n + D - N), \dots, \min(n, D)\}$
PMF	$\frac{\binom{D}{k} \binom{N-D}{n-k}}{\binom{N}{n}}$
CDF	Обобщённая гипергеометрическая функция
Среднее	$n \frac{D}{N}$
Мода	$\left\lfloor \frac{(n+1)(D+1)}{N+2} \right\rfloor$
Дисперсия	$n \frac{D}{N} \frac{N-D}{N} \frac{N-n}{N-1}$

Подробнее о методах для гипергеометрического распределения можно найти [здесь](#).

Распределение Пуассона

Распределение Пуассона значительно отличается от рассмотренных выше распределений своей «предметной» областью: теперь рассматривается не вероятность наступления того или иного исхода испытания, а интенсивность событий, то есть среднее количество событий в единицу времени.

Определение Распределение Пуассона описывает вероятность наступления k независимых событий при средней интенсивности событий λ . В данном случае λ — безразмерная величина, равная произведению времени наблюдения и частоты возникновения событий.

Пример

Рассмотрим колл-центр, который случайным образом принимает в среднем $\lambda=3$ звонка в минуту в любое время суток. Если вызовы независимы, получение одного из них не меняет вероятности того, когда поступит следующий. Согласно этим предположениям, количество k вызовов, полученных в течение любой минуты, имеет распределение вероятностей Пуассона. Тогда вероятность получения $k=$ от 1 до 4 вызовов составляет около 0.77, в то время как вероятность получения 0 или хотя бы 5 вызовов составляет около 0.23.

Классическим примером, используемым для обоснования распределения Пуассона, является количество событий радиоактивного распада за фиксированный период наблюдения.

Посмотрим, как графически показать PMF и CDF распределения Пуассона в Python. Функция PMF распределения Пуассона показывает вероятность наступления ровно k

событий: $P(X = k) = \frac{\lambda^k}{k!} e^{-\lambda}$

```
import numpy as np
from scipy.stats import poisson
import matplotlib.pyplot as plt

# Создаем поле для графика
plt.figure(figsize=(15, 10))

# Определяем параметры
x = np.arange(0, 15, 1)
plt.scatter(x, poisson.pmf(x, 0.4), c='b', label='mu = 0.4')
plt.scatter(x, poisson.pmf(x, 1), c='r', label='mu = 1')
plt.scatter(x, poisson.pmf(x, 4), c='g', label='mu = 4')
plt.scatter(x, poisson.pmf(x, 10), c='black', label='mu = 10')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='upper right')

# Отображаем график
plt.title("PMF распределения Пуассона")
plt.xlabel("Количество событий (k)")
plt.ylabel("Вероятность")
plt.show()
```

Функция CDF показывает накопленную вероятность для значений

$$F(X \leq k) = \sum_{i=0}^k P(X = i)$$

```
import numpy as np
from scipy.stats import poisson
import matplotlib.pyplot as plt

# Создаем поле для графика
plt.figure(figsize=(15, 10))

# Определяем параметры
x = np.arange(0, 15, 0.01)
plt.plot(x, poisson.cdf(x, 0.4), c='b', label='mu = 0.4')
plt.plot(x, poisson.cdf(x, 1), c='r', label='mu = 1')
plt.plot(x, poisson.cdf(x, 4), c='g', label='mu = 4')
plt.plot(x, poisson.cdf(x, 10), c='black', label='mu = 10')

# Добавляем сетку и легенду
plt.grid()
plt.legend(loc='lower right')

# Отображаем график
plt.title("CDF распределения Пуассона")
plt.xlabel("Количество событий (k)")
plt.ylabel("Накопленная вероятность")
```



```
plt.show()
```

Основные параметры распределения Пуассона:

Параметр	Значение
Обозначение	$\text{Pois}(\lambda)$
Параметры	$\lambda \in (0; \infty)$
Выборка — количество событий	$k \in N_0$
PMF	$\frac{\lambda^k}{k!} e^{-\lambda}$
CDF	Регуляризованная гамма-функция
Среднее	λ
Мода	$[\lambda]$
Медиана	$\approx \left\lceil \lambda + \frac{1}{3} - \frac{1}{50\lambda} \right\rceil$
Дисперсия	λ

Подробнее о методах для распределения Пуассона можно найти [здесь](#).

Задачи

В представленном [Collab ноутбуке](#) приведен набор задач по генерации данных по рассмотренным распределениям. Попробуйте их решить, также можете самостоятельно попрактиковаться в визуализации распределений, используя примеры кода в материалах этого юнита.

- [scipy.stats.randint](#)
- [scipy.stats.bernoulli](#)
- [scipy.stats.binom](#)
- [scipy.stats.geom](#)
- [scipy.stats.nbinom](#)
- [scipy.stats.hypergeom](#)
- [scipy.stats.poisson](#)

Урок 5. Абсолютно непрерывные распределения вероятностей

В этом уроке мы рассмотрим непрерывный тип данных и основные распределения. Определение Абсолютно непрерывное распределение вероятностей — это распределение вероятностей для действительных чисел с бесконечным количеством возможных значений. Поскольку значений бесконечно много, вероятность получения каждого конкретного значения равна нулю.

$$P(X = c) = \lim_{\epsilon \rightarrow 0} P(c \leq X \leq c + \epsilon) = \int_c^{c+\epsilon} f(x)dx = 0.$$

Это аналогично попытке выбрать одну точку на прямой — математически возможно, но практическая вероятность нулевая. Поэтому для абсолютно непрерывных случаев принято говорить не о вероятности конкретного значения, а о функции плотности вероятности (probability density function, PDF) на конечном интервале и, соответственно, о вероятности получения числа из этого интервала.

Определение Реальная случайная величина X имеет абсолютно непрерывное распределение вероятностей, если существует функция $f: \mathbb{R} \rightarrow [0; \infty)$ такая, что для каждого интервала $I = [a, b] \subset \mathbb{R}$ вероятность X принадлежности к I задаётся интегралом от f по

$$P(a \leq X \leq b) = \int_a^b f(x)dx. \text{ Если интервал } [a, b] \text{ заменить любым измеримым набором } A,$$

соответствующее равенство остаётся в силе: $P(X \in A) = \int_A f(x)dx.$

Абсолютно непрерывное равномерное распределение

Определение В теории вероятностей и статистике непрерывные равномерные распределения (или прямоугольные распределения) представляют собой семейство симметричных распределений вероятностей. Такое распределение описывает эксперимент, в котором имеется произвольный результат, лежащий в определённых границах. Границы определяются параметрами a и b , которые являются минимальным и максимальным значениями. Интервал может быть закрытым — $[a, b]$ или открытым — (a, b) . Значения f в точках a и b обычно не важны, поскольку не влияют на расчёт самой

вероятности. Иногда они выбираются равными нулю, а иногда $\frac{1}{b-a}$. Последнее уместно в контексте оценки методом максимального правдоподобия. В контексте анализа Фурье можно принять значения f в точках a и b равными $\frac{1}{2(b-a)}$.

Функция плотности:

$$f(x) = \begin{cases} \frac{1}{b-a}, & x \in [a; b] \\ 0, & x \notin [a; b] \end{cases}$$

Функция распределения:

$$F(x) = \begin{cases} 0, & x < a \\ \frac{x-a}{b-a}, & a \leq x \leq b \\ 1, & x > b \end{cases}$$

Посмотрим, как графически показать PDF и CDF абсолютно непрерывного равномерного распределения в Python.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import uniform

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))
x = np.linspace(-1, 11, 5000)
ax1.plot(x, uniform.pdf(x), 'b', label='U(0,1)')
ax1.plot(x, uniform.pdf(x, loc=2, scale=5), 'r', label='U(2,7)')
ax1.plot(x, uniform.pdf(x, loc=8, scale=2), 'g', label='U(8,10)')
ax1.set_title('Функция плотности (PDF)')
ax1.grid(True)

ax2.plot(x, uniform.cdf(x), 'b', label='U(0,1)')
ax2.plot(x, uniform.cdf(x, loc=2, scale=5), 'r', label='U(2,7)')
ax2.plot(x, uniform.cdf(x, loc=8, scale=2), 'g', label='U(8,10)')
ax2.set_title('Функция распределения (CDF)')
ax2.grid(True)
for ax in (ax1, ax2):
    ax.legend()
    ax.set_xlabel('x')
    ax.set_ylabel('f(x) / F(x)')
plt.show()
```

Параметр	Значение
Обозначение	$U_{[a,b]}$
Параметры	$-\infty < a < b < +\infty$
Выборка - множество возможных значений	$[a,b]$
PDF	$\frac{1}{b-a}, a \leq x \leq b$ $0, x < a \text{ или } x > b$
CDF	$0, x < a$ $\frac{x-a}{b-a}, a \leq x \leq b$ $1, x > b$
Среднее	$\frac{b+a}{2}$
Мода	-
Медиана	$\frac{b+a}{2}$
Дисперсия	$\frac{(b-a)^2}{12}$

Подробнее о методах для непрерывного равномерного распределения можно найти [здесь](#).
 Распределение Гаусса (нормальное распределение)

Нормальное распределение хорошо моделирует величины, описывающие природные явления, шумы термодинамической природы и погрешности измерений.

Кроме того, согласно центральной предельной теореме, сумма большого количества независимых слагаемых одного порядка сходится к нормальному распределению, независимо от распределений слагаемых. Благодаря этому свойству, нормальное распределение популярно в статистическом анализе, и многие статистические тесты рассчитаны на нормально распределённые данные.

Определение Плотность вероятности нормального распределения с параметрами μ и σ описывается функцией Гаусса. Если $\mu=0$ и $\sigma=1$, то такое распределение называется стандартным.

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Функция распределения:

$$F(x) = \frac{1}{2} \left[1 + \operatorname{erf} \left(\frac{x - \mu}{\sigma \sqrt{2}} \right) \right]$$

где erf — функция ошибок.

Посмотрим, как графически показать PDF и CDF нормального распределения в Python.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

plt.figure(figsize=(15, 6))
x = np.linspace(-10, 10, 1000)
plt.subplot(121)
for params in [(0,1), (5,3), (-5,0.5)]:
    plt.plot(x, norm.pdf(x, *params), label=f'μ={params[0]},
σ={params[1]}')
plt.title('Функция плотности (PDF)')
plt.grid(True)
plt.subplot(122)
for params in [(0,1), (5,3), (-5,0.5)]:
    plt.plot(x, norm.cdf(x, *params), label=f'μ={params[0]},
σ={params[1]}')
plt.title('Функция распределения (CDF)')
plt.grid(True)
plt.legend()
plt.show()
```

Основные параметры нормального распределения:

Параметр	Значение
Обозначение	$N(\mu, \sigma^2)$
Параметры	$\mu \in \mathbb{R}$ $\sigma^2 \in \mathbb{R}$
Выборка — множество возможных значений	$x \in \mathbb{R}$

PDF	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
CDF	$\frac{1}{2} \left(1 + \operatorname{erf}\left(\frac{x-\mu}{\sqrt{2}\sigma}\right)\right)$
Среднее	μ
Мода	μ
Медиана	μ
Дисперсия	σ^2

Подробнее о методах для распределения Гаусса можно найти [здесь](#).

Гауссовы распределения обладают некоторыми уникальными свойствами, которые ценны в аналитических исследованиях.

Например, любая линейная комбинация фиксированного набора независимых нормальных распределений является нормальным распределением. Многие результаты и методы, такие как метод наименьших квадратов, могут быть получены аналитически в явном виде, когда соответствующие переменные распределены нормально.

На бесконечной делимости нормального распределения основан (z)-тест. Этот тест используется для проверки равенства математического ожидания выборки нормально распределённых величин некоторому значению. Значение дисперсии должно быть известно. Если значение дисперсии неизвестно и рассчитывается на основании анализируемой выборки, то применяется (t)-тест, основанный на распределении Стьюдента.

Нужно выделить несколько распределений, тесно связанных с нормальным:

- линейная комбинация нормальных распределений — тоже нормальное распределение;
- сумма квадратов величин, распределённых по стандартному нормальному закону — это распределение «хи-квадрат» χ^2 ;
- отношение величин, распределённых по «хи-квадрат», распределено по Фишеру;
- отношение величин, распределённых нормально, распределено по Коши;
- линейная комбинация величин, распределённых по Коши, тоже распределена по Коши;
- отношение нормально распределённой случайной величины к величине, распределённой по «хи-квадрат», будет распределено по Стьуденту.

Экспоненциальное распределение

Определение Экспоненциальное (показательное) распределение описывает интервалы времени между независимыми событиями, происходящими со средней интенсивностью λ . Количество наступлений таких событий за некоторый отрезок времени описывается дискретным распределением Пуассона.

Кроме теории надёжности, экспоненциальное распределение применяется для описания социальных явлений, в экономике, в теории массового обслуживания, в транспортной логистике — везде, где необходимо моделировать поток событий.

Функция плотности:

$$f(x) = \begin{cases} \lambda e^{-\lambda x}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Функция распределения:

$$F(x) = 1 - e^{-\lambda x}, x \geq 0$$

Дискретный вариант экспоненциального распределения — это геометрическое распределение.

Посмотрим, как графически показать PDF и CDF экспоненциального распределения в Python.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import expon

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
x = np.linspace(0, 5, 500)
for scale in [1, 1/0.2, 1/2]:
    ax1.plot(x, expon.pdf(x, scale=scale), label=f'λ={1/scale:.1f}')
ax1.set_title('Функция плотности (PDF)')
for scale in [1, 1/0.2, 1/2]:
    ax2.plot(x, expon.cdf(x, scale=scale), label=f'λ={1/scale:.1f}')
ax2.set_title('Функция распределения (CDF)')
for ax in (ax1, ax2):
    ax.legend()
    ax.grid(True)
plt.show()
```

Основные параметры экспоненциального распределения:

Параметр	Значение
Обозначение	$E_{\lambda}(x)$
Параметры	$\lambda > 0$
Выборка — множество возможных значений	$x \in [0; \infty)$
PDF	$\lambda e^{-\lambda x}$
CDF	$1 - e^{-\lambda x}$
Среднее	$\frac{1}{\lambda}$
Мода	0
Медиана	$\frac{\ln 2}{\lambda}$
Дисперсия	$\frac{1}{\lambda^2}$

Подробнее о методах для экспоненциального распределения можно найти [здесь](#).

Определение Если функцию плотности вероятностей экспоненциального распределения отразить зеркально в область отрицательных значений, то получится распределение Лапласа, также называемое двойным экспоненциальным.

Благодаря более тяжёлым хвостам, чем у нормального распределения, распределение Лапласа используется для моделирования некоторых видов погрешностей измерений в энергетике, а также находит применение в физике, экономике, финансовой статистике, телекоммуникации и т. д.

Функция плотности:

$$f(x) = \frac{1}{2b} e^{-\frac{|x-\mu|}{b}}$$

Функция распределения:

$$f(x) = \frac{1}{2b} e^{-\frac{|x-\mu|}{b}}$$

Посмотрим, как графически показать PDF и CDF распределения Лапласа в Python.

```
from scipy.stats import laplace
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
x = np.linspace(-10, 10, 500)
for (loc, scale) in [(0,1), (5,1), (0,4), (-5,0.5)]:
    ax1.plot(x, laplace.pdf(x, loc, scale), label=f'μ={loc}, b={scale}')
ax1.set_title('Функция плотности (PDF)')
for (loc, scale) in [(0,1), (5,1), (0,4), (-5,0.5)]:
    ax2.plot(x, laplace.cdf(x, loc, scale), label=f'μ={loc}, b={scale}')
ax2.set_title('Функция распределения (CDF)')
for ax in (ax1, ax2):
    ax.legend()
    ax.grid(True)
plt.show()
```

Основные параметры распределения Лапласа:

Параметр	Значение
Обозначение	$L_{\mu,b}(x)$
Параметры	$\mu \in \mathbb{R}$ $b > 0$
Выборка — множество возможных значений	$x \in (-\infty; +\infty)$
PDF	$\frac{1}{2b} e^{-\frac{ x-\mu }{b}}$
CDF	$\frac{1}{2} e^{-\frac{x-\mu}{b}}, x \leq \mu$ $1 - \frac{1}{2} e^{-\frac{x-\mu}{b}}, x \geq \mu$
Среднее	μ

Мода	μ
Медиана	μ
Дисперсия	$2b^2$

Подробнее о методах для распределения Лапласа можно найти [здесь](#).

Определение Абсолютно непрерывное распределение вероятностей — это распределение вероятностей для действительных чисел с бесконечным количеством возможных значений. Поскольку значений бесконечно много, вероятность получения каждого конкретного значения равна нулю.

$$P(X = c) = \lim_{\epsilon \rightarrow 0} P(c \leq X \leq c + \epsilon) = \int_c^c f(x) dx = 0$$

Это аналогично попытке выбрать одну точку на прямой — математически возможно, но практическая вероятность нулевая. Поэтому для абсолютно непрерывных случаев принято говорить не о вероятности конкретного значения, а о функции плотности вероятности (probability density function, PDF) на конечном интервале и, соответственно, о вероятности получения числа из этого интервала.

Определение Реальная случайная величина X имеет абсолютно непрерывное распределение вероятностей, если существует функция $f: \mathbb{R} \rightarrow [0; \infty)$ такая, что для каждого интервала $I = [a, b] \subset \mathbb{R}$ вероятность X принадлежности к I задаётся интегралом от f по I :

$$P(a \leq X \leq b) = \int_a^b f(x) dx$$

Если интервал $[a, b]$ заменить любым измеримым набором A , соответствующее равенство остаётся в силе:

$$P(X \in A) = \int_A f(x) dx$$

Распределение Парето

Определение Распределение Парето представляет собой степенное распределение вероятностей, которое используется при описании социальных, управленческих, научных, геофизических, актуарных и многих других типов наблюдаемых явлений.

Принцип, первоначально применявшийся для описания распределения богатства в обществе, соответствует тенденции, согласно которой большая часть богатства принадлежит небольшой части населения. Этот принцип привёл к возникновению «принципа Парето» или «правило 80–20», утверждающего, что 80% результатов обусловлены 20% причин.

Распределения Парето со значением формы $\alpha = \log_4 5 \approx 1.16$ точно отражают это.

Эмпирические наблюдения показали, что распределение 80–20 подходит для широкого круга случаев, включая природные явления и деятельность человека.

Следующие примеры иногда рассматриваются как приблизительно распределённые по Парето

Пример

- все четыре переменные бюджетного ограничения домохозяйства: потребление, трудовой доход, доход от капитала и богатство;
- размеры населённых пунктов (несколько городов, много деревень);
- распределение интернет-трафика по размеру файлов, использующего протокол TCP (много файлов меньшего размера, несколько файлов большего размера);

- частота ошибок на жёстком диске;
- скопления конденсата Бозе–Эйнштейна вблизи абсолютного нуля;
- значения запасов нефти на нефтяных месторождениях (несколько крупных, много небольших);
- распределение по длине заданий, назначенных суперкомпьютерам (несколько больших и много маленьких);
- стандартизированная доходность по отдельным акциям;
- размеры частиц песка;
- размер метеоритов;
- серьёзность крупных потерь от несчастных случаев в определённых сферах бизнеса, таких как общая ответственность, коммерческие автомобили и компенсация работникам;
- количество времени, которое пользователь в Steam будет тратить на разные игры (в некоторые игры играют часто, но в большинство — почти никогда).

Функция плотности:

$$f(x) = \begin{cases} \frac{\alpha x_m^\alpha}{x^{\alpha+1}}, & x \geq x_m \\ 0, & x < x_m \end{cases}$$

Функция распределения:

$$F(x) = 1 - \left(\frac{x_m}{x}\right)^\alpha, x \geq x_m$$

К особенностям распределения можно отнести бесконечную дисперсию при $\alpha \leq 2$ и бесконечное среднее при $\alpha \leq 1$.

Посмотрим, как графически показать PDF и CDF распределения Парето в Python.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import pareto

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# PDF
x = np.linspace(1, 10, 1000)
for (a, xm) in [(1,1), (2,1), (1,4), (0.5,1)]:
    ax1.plot(x, pareto.pdf(x, a, scale=xm),
             label=f'α={a}, xm={xm}')
ax1.set_title('Функция плотности (PDF)')
ax1.set_ylim(0, 2.5)

# CDF
for (a, xm) in [(1,1), (2,1), (1,4), (0.5,1)]:
    ax2.plot(x, pareto.cdf(x, a, scale=xm),
             label=f'α={a}, xm={xm}')
ax2.set_title('Функция распределения (CDF)')

for ax in (ax1, ax2):
```

```
ax.legend()
ax.grid(True)
plt.show()
```

Основные параметры распределения Парето:

Параметр	Значение
Обозначение	$Pareto_{\alpha, x_m}(x)$
Параметры	$\alpha > 0$
Выборка — множество возможных значений	$x \in [x_m, \infty), x \in [x_m; +\infty)$
PDF	$\frac{\alpha x_m^\alpha}{x^{\alpha+1}}$
CDF	$1 - (\frac{x_m}{x})^\alpha$
Среднее	$\frac{\alpha x_m}{\alpha - 1}, \alpha > 1$
Мода	x_m
Медиана	$x_m \sqrt[\alpha]{2}$
Дисперсия	$\frac{\alpha x_m^2}{(\alpha - 1)^2(\alpha - 2)}$

! Важно: Подробнее о методах для распределения Парето можно найти [здесь](#).

Распределение Рэлея

Определение Распределение Рэлея часто наблюдается, когда общая величина вектора на плоскости связана с его направленными компонентами.

Один из примеров, когда распределение Рэлея возникает естественным образом, — когда скорость ветра анализируется в двух измерениях. Предполагая, что каждый компонент не коррелирован, нормально распределён с равной дисперсией и нулевым средним значением, общая скорость ветра будет характеризоваться распределением Рэлея.

Второй пример распределения возникает в случае случайных комплексных чисел, действительная и мнимая составляющие которых распределены независимо и идентично по гауссову с равной дисперсией и нулевым средним. В этом случае абсолютное значение комплексного числа распределено по Рэлею.

Применение оценки σ можно найти в магнитно-резонансной томографии (МРТ). Поскольку изображения МРТ записываются как сложные изображения, но чаще всего представляются как изображения магнитуды, фоновые данные распределены по Рэлею. Следовательно, приведённую выше формулу можно использовать для оценки дисперсии шума на МРТ-изображении по фоновым данным.

Распределение Рэлея также использовалось в области питания для изучения питательных веществ при конкретных диетах и реакций человека и животных. Таким образом, параметр σ может использоваться для расчёта зависимости реакции на питательные вещества.

В области баллистики распределение Рэля используется для вычисления вероятной круговой ошибки — показателя точности оружия.

В физической океанографии распределение высоты волн приблизительно соответствует распределению Рэля.

Функция плотности:

$$f(x) = \begin{cases} \frac{x}{\sigma^2} e^{-\frac{x^2}{2\sigma^2}}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Функция распределения:

$$F(x) = 1 - e^{-\frac{x^2}{2\sigma^2}}, x \geq 0$$

Определение Особенности: нулевая плотность в точке 0; мода = σ ; быстрое убывание "хвостов".

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import rayleigh

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# PDF
x = np.linspace(0, 5, 500)
for scale in [0.5, 1, 2, 3]:
    ax1.plot(x, rayleigh.pdf(x, scale=scale),
             label=f'σ={scale}')
ax1.set_title('Функция плотности (PDF)')
ax1.set_ylim(0, 1.2)

# CDF
for scale in [0.5, 1, 2, 3]:
    ax2.plot(x, rayleigh.cdf(x, scale=scale),
             label=f'σ={scale}')
ax2.set_title('Функция распределения (CDF)')

for ax in (ax1, ax2):
    ax.legend()
    ax.grid(True)
plt.show()
```

Параметр	Значение
Обозначение	$R_{\sigma}(x)$
Параметры	$\sigma > 0$
Выборка — множество возможных значений	$x \in [0; +\infty)$
PDF	$\frac{x}{\sigma^2} e^{-\frac{x^2}{2\sigma^2}}$

CDF	$1 - e^{-\frac{x^2}{2\sigma^2}}$
Среднее	$\sigma \sqrt{\frac{\pi}{2}}$
Мода	σ
Медиана	$\sigma \sqrt{2 \ln 2}$
Дисперсия	$\frac{4 - \pi}{2} \sigma^2$

! Важно: Подробнее о методах для распределения Рэлея можно найти [здесь](#).

Задачи

В представленном [Collab ноутбуке](#) приведен набор задач по генерации данных по рассмотренным непрерывным распределениям. Попробуйте их решить, также можете самостоятельно попрактиковаться в визуализации распределений, используя примеры кода в материалах этого юнита.

[scipy.stats.uniform](#)
[scipy.stats.norm](#)
[scipy.stats.expon](#)
[scipy.stats.pareto](#)
[scipy.stats.rayleigh](#)

Интервальные оценки параметров распределения. Проверка гипотез

Урок 1. Введение в модуль

Этот модуль дает фундаментальные знания и практические навыки, необходимые для анализа данных, статистического моделирования и проверки гипотез. В модуле вы найдете математические конспекты тем. Ознакомьтесь с этими конспектами, записями занятий и задавайте вопросы преподавателю на лекциях.

Проверка статистических гипотез является ключевым компонентом для понимания и оценки моделей в машинном обучении. В этом модуле вы познакомитесь с методами проверки гипотез, которые помогут принимать обоснованные решения при разработке и тестировании моделей. От предобработки данных до оценки эффективности алгоритмов и улучшения их параметров -- статистические гипотезы дают возможность проверить, действительно ли модель выполняет свои задачи так, как задумано, или результаты получены случайно.

Использование проверки гипотез позволяет выявить значимые особенности и отфильтровать «шум», что особенно важно в условиях, когда данные неоднородны или подвержены внешним влияниям. Также проверка гипотез помогает понять, какие изменения в данных, параметрах модели или самой архитектуре действительно ведут к улучшению качества, а какие лишь кажутся полезными. Проверка гипотез используется для объективной оценки результатов и понимания границ применимости методов, создавая основу для построения надежных и устойчивых моделей машинного обучения.

В этом модуле вы рассмотрите теоретические математические основы проверки гипотез, чтобы далее научиться применять методы на практике.

По завершении модуля вы сможете:

Понимать и использовать доверительные интервалы. Вы научитесь рассчитывать доверительные интервалы для различных параметров, что позволит уверенно определять точность своих оценок и понимать их ограничения.

Анализировать распределения данных. Полученные знания позволят анализировать и интерпретировать распределения данных с использованием хи-квадрат, распределений Стюдента и Фишера, а также трансформаций нормальных выборок.

Строить статистические модели. Вы освоите методы построения и применения статистических моделей, основанных на распределении нормальных выборок и преобразовании случайных величин.

Проверять статистические гипотезы. Модуль познакомит вас с методологией проверки гипотез, включая ошибки I и II рода, и научит применять соответствующие статистические критерии на практике.

Работать с параметрическими и непараметрическими критериями согласия. Обучение охватывает методы проверки согласия, что станет ценным инструментом при проверке параметрических гипотез о распределении данных.

Использовать статистические методы в междисциплинарных исследованиях. Знания, полученные в рамках модуля, будут полезны в различных прикладных исследованиях, где требуются количественные доказательства и оценка надежности результатов.

Изучив темы данного модуля, вы постепенно сформируете аналитический инструмент для оценки, проверки и построения выводов на основе данных.

Урок 2. Доверительные интервалы

В этом уроке мы познакомимся с понятием доверительного интервала, откуда он берётся, как его строить в разных типичных ситуациях, как интерпретировать и проверять. Также научимся применять теорию на практике с помощью Python: строить нормальные и t -интервалы, χ^2 -интервалы для дисперсии, бутстрэп и интервалы для долей.

- Доверительные интервалы используются для оценки неизвестных параметров, таких как среднее, доля или дисперсия.
- Доверительный интервал дает диапазон значений, в котором неизвестный параметр попадает с заданной частотой при повторении выборок.
- Практически все точные ДИ строятся с использованием функции $G(X, \theta)$, распределение которой не зависит от θ .
- Интервалы для дисперсии χ^2 : в случае нормальной выборки существует точное соотношение между выборочной и генеральной дисперсией.
- Интервалы для доли: во многих задачах нас интересует не среднее или дисперсия, а доля успехов - т.е. вероятность наступления события.
- Бутстрэп-интервалы: иногда распределение статистики слишком сложно, чтобы вывести для него формулу доверительного интервала.
- Выбор уровня доверия и ширины интервала: при построении доверительного интервала важно понимать, насколько широкий он получится и какая выборка нужна, чтобы достичь желаемой точности.

Доверительный интервал для среднего нормальной популяции

Реализуем описанные выкладки на Python:

```
import numpy as np
from scipy.stats import norm

np.random.seed(1)
n = 25
mu_true = 5.0
sigma = 2.0 # известно
data = np.random.normal(loc=mu_true, scale=sigma, size=n)
xbar = data.mean()
alpha = 0.05
z = norm.ppf(1 - alpha/2)
ci_lower = xbar - z * sigma / np.sqrt(n)
ci_upper = xbar + z * sigma / np.sqrt(n)
print(f"̄x = {xbar:.3f}, {100*(1-alpha):.0f}% CI = [{ci_lower:.3f}, {ci_upper:.3f}]")
```

Когда генеральная дисперсия σ^2 неизвестна — что в реальности происходит почти всегда, её приходится оценивать по выборке. В этом случае распределение статистики уже нельзя считать строго нормальным, оно будет подчиняться распределению Стьюдента — и доверительный интервал будет строиться на его основе.

Также реализуем на Python:

```
from scipy.stats import t
```

```
s = data.std(ddof=1)
t_q = t.ppf(1 - alpha/2, df=n-1)
ci_t = (xbar - t_q * s / np.sqrt(n), xbar + t_q * s / np.sqrt(n))
print(f"100*(1-alpha):.0f}% t-CI = [{ci_t[0]:.3f}, {ci_t[1]:.3f}]")
```

Интервалы для дисперсии χ^2

Реализуем интервалы для дисперсии на Python:

```
from scipy.stats import chi2

chi2_low = chi2.ppf(alpha/2, df=n-1)
chi2_high = chi2.ppf(1-alpha/2, df=n-1)
ci_var = ((n-1)*s*s / chi2_high, (n-1)*s*s / chi2_low)
ci_sigma = (np.sqrt(ci_var[0]), np.sqrt(ci_var[1]))
print(f"CI for sigma^2: [{ci_var[0]:.3f}, {ci_var[1]:.3f}]")
print(f"CI for sigma: [{ci_sigma[0]:.3f}, {ci_sigma[1]:.3f}]")
```

Интервалы для доли

Реализуем интервалы для доли на Python — встроенных методов в `scipy` на этот раз нет, поэтому сделаем расчёт вручную по формулам:

```
import numpy as np
from scipy.stats import norm

k = 45      # число успехов
n = 100     # число испытаний
alpha = 0.05 # уровень значимости

# Квантиль нормального распределения
z = norm.ppf(1 - alpha/2)

# Wald-интервал
p_hat = k / n
wald_lower = p_hat - z * np.sqrt(p_hat * (1 - p_hat) / n)
wald_upper = p_hat + z * np.sqrt(p_hat * (1 - p_hat) / n)

# Agresti-Coull-интервал
n_tilde = n + z**2
p_tilde = (k + z**2 / 2) / n_tilde
agresti_lower = p_tilde - z * np.sqrt(p_tilde * (1 - p_tilde) / n_tilde)
agresti_upper = p_tilde + z * np.sqrt(p_tilde * (1 - p_tilde) / n_tilde)
```

Бутстрэп-интервалы

Реализуем бутстрэп на Python:

```
import numpy as np

def bootstrap_ci(data, statfunc=np.mean, B=2000, alpha=0.05):
    n = len(data)
    boot_stats = np.empty(B)
    for i in range(B):
        sample = np.random.choice(data, size=n, replace=True)
        boot_stats[i] = statfunc(sample)
```

```
lo = np.quantile(boot_stats, alpha/2)
hi = np.quantile(boot_stats, 1-alpha/2)
return lo, hi
```

```
ci_boot = bootstrap_ci(data, statfunc=np.mean, B=5000)
print("Bootstrap percentile CI:", ci_boot)
```

Выбор уровня доверия и ширины интервала

Урок 3. Распределения, связанные с нормальным

Мы познакомимся с тремя классическими распределениями, без которых не обходится почти ни один статистический анализ — хи-квадрат χ^2 , Стьюдента (t) и Фишера (F). Это настоящие «рабочие лошадки» статистики: каждое из них возникает естественным образом из нормального распределения и лежит в основе доверительных интервалов, t-тестов и дисперсионного анализа (ANOVA).

Мы разберём, откуда эти распределения появляются, в каких задачах они используются и почему их формы зависят от числа степеней свободы.

Также вы научитесь симулировать их в Python, строить доверительные интервалы и применять их для реальных задач — например, оценивать дисперсии и сравнивать группы.

- Рассмотрены три классических распределения: хи-квадрат, Стьюдента и Фишера.
- Хи-квадрат используется для оценки и проверки дисперсий, а также для построения доверительных интервалов.
- t-распределение применяется для t-тестов и t-интервалов, а F-распределение используется в задачах сравнения разбросов и в ANOVA.
- Важно помнить о нормальности данных и о том, что t-тест и F-тест чувствительны к отклонениям от нормальности.
- Для проверки равенства дисперсий при ненормальности рекомендуется использовать альтернативные методы, такие как тест Левена или Брауна-Форсайта.

χ^2 (хи-квадрат)

Рассмотрим симуляцию Python и сравним её с теорией

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import chi2

np.random.seed(0)
k = 5
B = 20000
# симулируем сумму квадратов k стандартных нормалей
chis = np.sum(np.random.normal(size=(B, k))**2, axis=1)

# гистограмма и теоретическая плотность
xs = np.linspace(0, np.percentile(chis, 99.5), 500)
plt.hist(chis, bins=80, density=True, alpha=0.5, label='симуляция')
plt.plot(xs, chi2.pdf(xs, df=k), label=f'$\\chi^2(df={k}) pdf', lw=2)
plt.legend(); plt.xlabel('x'); plt.ylabel('density'); plt.title('$\\chi^2$ -- симуляция vs теоретика')
plt.show()
```

Распределение Стьюдента (t)

Аналогично рассмотрим симуляцию на Python и сравним t-распределения с нормальным:


```

from scipy.stats import t, norm

np.random.seed(1)
df = 5
B = 20000
# симуляция t как ratio
z = np.random.normal(size=B)
chi = np.sum(np.random.normal(size=(B, df))**2, axis=1)
t_sim = z / np.sqrt(chi/df)

xs = np.linspace(-6, 6, 400)
plt.hist(t_sim, bins=120, density=True, alpha=0.4, label='симуляция t')
plt.plot(xs, t.pdf(xs, df), label=f't pdf df={df}', lw=2)
plt.plot(xs, norm.pdf(xs), label='N(0,1) pdf', lw=1, ls='--')
plt.legend(); plt.title('t-распределение (симуляция) и нормаль'); plt.show()

```

Распределение Фишера (F)

Рассмотрим симуляцию на Python и применение для теста равенства дисперсий:

```

from scipy.stats import f

np.random.seed(2)
n1, n2 = 30, 40
# симуляция нормальных выборок с разными дисперсиями
s1 = np.random.normal(loc=0, scale=1.2, size=n1)
s2 = np.random.normal(loc=0, scale=0.9, size=n2)

var1 = s1.var(ddof=1)
var2 = s2.var(ddof=1)
F_stat = var1 / var2
df1, df2 = n1-1, n2-1
p_upper = 1 - f.cdf(F_stat, df1, df2)
p_two_sided = 2 * min(p_upper, 1-p_upper) # грубая двухсторонняя версия
print(f"F = {F_stat:.3f}, df1={df1}, df2={df2}, one-sided p = {p_upper:.4f}, approx two-sided p={p_two_sided:.4f}")

```

Урок 4. Преобразования нормальных выборок

В этом уроке мы разберёмся, как ведёт себя нормальное распределение при линейных и ортогональных преобразованиях. Почему любая линейная комбинация нормальных величин остаётся нормальной? Когда новые координаты сохраняют независимость? Как из этих свойств вырастают знакомые результаты — независимость выборочного среднего и остатков, χ^2 -разложение суммы квадратов и многое другое?

- Линейные комбинации нормальных величин также являются нормальными.
- Ортогональные матрицы сохраняют евклидову норму и независимость стандартных нормальных векторов.
- Ортогональные преобразования позволяют выделить компоненты, такие как направление среднего и ортогональные остатки.
- Линейные преобразования сохраняют нормальность данных, а ортогональные — независимость компонентов.
- Плотность стандартного нормала радиально симметрична и не меняется при поворотах и отражениях.
- Типичные ошибки: неверное предположение о независимости компонентов при нулевой ковариации и использование ортогональных матриц для данных, отличных от стандартных нормальных.

- В случае ненормальных данных, можно использовать приближения центральной предельной теоремы или робастные методы, такие как бутстрэп и перестановочные тесты.

Линейные преобразования сохраняют нормальность

Проверим, что линейное преобразование нормальных данных остаётся нормальным при помощи Python.

```
import numpy as np
from scipy.stats import normaltest

np.random.seed(1)
X = np.random.normal(size=(1000, 2)) # 2D нормальные
A = np.array([[2, 1], [-1, 3]])      # произвольная матрица
Y = X @ A.T                          # линейное преобразование

print("Проверка нормальности компонент Y:")
for i in range(2):
    stat, p = normaltest(Y[:, i])
    print(f"Компонента {i+1}: p-value = {p:.3f}")
```

Ортогональные преобразования сохраняют независимость

```
from numpy.linalg import qr
import numpy as np

n = 5
A = np.random.normal(size=(n, n))
Q, _ = qr(A) # получаем ортогональную матрицу
X = np.random.normal(size=n)
Y = Q @ X

print("Среднее Y:", np.mean(Y).round(3))
print("Дисперсия Y:", np.var(Y, ddof=1).round(3))
print("Проверим ортогональность Q:", np.allclose(Q @ Q.T, np.eye(n)))
```

Разложение на среднее и остатки

Реализуем на Python:

```
from numpy.linalg import qr
from scipy.stats import chi2
import numpy as np

n = 5
A = np.random.normal(size=(n, n))
Q, _ = qr(A) # получаем ортогональную матрицу
X = np.random.normal(size=n)
Y = Q @ X

print("Среднее Y:", np.mean(Y).round(3))
print("Дисперсия Y:", np.var(Y, ddof=1).round(3))
print("Проверим ортогональность Q:", np.allclose(Q @ Q.T, np.eye(n)))
scaled = (n-1)*np.var(X, ddof=1) # иллюстративная шкалировка
print("CDF  $\chi^2$ :", chi2.cdf(scaled, df=n-1))
```

Проекция и матричная форма

Реализуем на Python:

```
import numpy as np

n = 5
X = np.random.normal(0, 1, n)
P = np.ones((n, n)) / n
proj = P @ X
resid = X - proj

print("X:", X.round(2))
print("Проекция на среднее:", proj.round(2))
print("Остатки:", resid.round(2))
print("Проверка ортогональности:", np.isclose(np.dot(proj, resid), 0))
```

Урок 5. Доверительные интервалы для параметров нормального распределения

В этом уроке мы разберёмся, как из данных получить не просто одно число — «оценку параметра», — а диапазон значений, в котором с заданной уверенностью лежит истинное значение. Также научимся строить и интерпретировать доверительные интервалы для параметров нормального распределения: для среднего μ (в ситуациях, когда дисперсия σ^2 известна или оценивается по выборке), для самой дисперсии и стандартного отклонения.

- Доверительные интервалы для параметров нормального распределения позволяют определить диапазон значений, в котором находится истинное значение с заданной вероятностью.
- Существуют точные (не асимптотические) интервалы для нормальной модели, основанные на распределениях с известными функциональными связями.
- Интервалы для дисперсии и отношения дисперсий используют распределение хи-квадрат и F-распределение соответственно.
- Проверка покрытия через симуляцию помогает оценить, насколько хорошо формулы работают на практике.
- Нормальность данных является важным предположением для точных t- и χ^2 -интервалов.
- Малые выборки требуют использования бутстрэп-интервалов для обеспечения реалистичного покрытия.
- Чувствительность к выбросам может быть уменьшена с помощью робастных мер или бутстрэп-методов.

Интервал для среднего (μ)

Проверка покрытия через симуляцию

Формулы дают теоретическое покрытие при выполнении предпосылок. Для того чтобы понять, насколько хорошо они работают в практике (особенно при малых n), полезно симулировать множество выборок и оценить настоящую долю интервалов, содержащих параметр (empirical coverage). Ниже — пример для t- и z-интервалов (когда σ "известна" или оценивается).

Рассмотрим на Python симуляцию покрытия t-интервала

```
import numpy as np
from scipy.stats import t, norm
```

```

def empirical_coverage_t(mu=0, sigma=1, n=10, alpha=0.05, B=5000, seed=0):
    rng = np.random.default_rng(seed)
    contains = 0
    for _ in range(B):
        x = rng.normal(loc=mu, scale=sigma, size=n)
        xbar = x.mean()
        s = x.std(ddof=1)
        t_q = t.ppf(1-alpha/2, df=n-1)
        lo = xbar - t_q * s / np.sqrt(n)
        hi = xbar + t_q * s / np.sqrt(n)
        if lo <= mu <= hi:
            contains += 1
    return contains / B

print("Empirical coverage (n=10):", empirical_coverage_t(n=10))
print("Empirical coverage (n=30):", empirical_coverage_t(n=30))

```

Примеры на Python

Сгенерируем выборку и рассмотрим сквозные примеры применения всех изученных методов на Python:

```

import numpy as np
from scipy.stats import norm, t, chi2, f

np.random.seed(42)

# Параметры
n = 20
mu_true = 2.5
sigma_true = 1.7
alpha = 0.05

# сгенерируем выборку
x = np.random.normal(loc=mu_true, scale=sigma_true, size=n)
xbar = x.mean()
s = x.std(ddof=1)

# 1) z-interval (если sigma известно)
zq = norm.ppf(1-alpha/2)
ci_z = (xbar - zq * sigma_true/np.sqrt(n), xbar + zq * sigma_true/np.sqrt(n))
print("z-CI (sigma known):", ci_z)

# 2) t-interval (sigma unknown)
tq = t.ppf(1-alpha/2, df=n-1)
ci_t = (xbar - tq * s/np.sqrt(n), xbar + tq * s/np.sqrt(n))
print("t-CI (sigma unknown):", ci_t)

# 3) CI for sigma^2 via chi2
chi_low = chi2.ppf(alpha/2, df=n-1)
chi_high = chi2.ppf(1-alpha/2, df=n-1)
ci_var = ((n-1)*s*s/chi_high, (n-1)*s*s/chi_low)
ci_sigma = (np.sqrt(ci_var[0]), np.sqrt(ci_var[1]))
print("CI for sigma^2:", ci_var)
print("CI for sigma:", ci_sigma)

# 4) Prediction interval for a single future observation
pred_halfwidth = tq * s * np.sqrt(1 + 1/n)
pred_interval = (xbar - pred_halfwidth, xbar + pred_halfwidth)
print("Prediction interval for next obs:", pred_interval)

```

Урок 6. Асимптотический доверительный интервал

Когда выборка большая, мы можем позволить себе немного меньше «магии распределений» и больше — универсальных закономерностей. В этом уроке мы разберёмся, как Центральная предельная теорема превращает любую разумную оценку в почти нормальную, и научимся на этом строить асимптотические доверительные интервалы (или Wald-интервалы). Поймём, где они работают хорошо, а где стоит предпочесть бутстрэп или преобразования, и как оценивать качество (покрытие) таких интервалов через симуляции.

- Центральная предельная теорема превращает любую разумную оценку в почти нормальную.
- Асимптотические доверительные интервалы (Wald-интервалы) строятся на основе оценки асимптотической стандартной ошибки.
- Погрешности могут быть существенными для малых n , поэтому ключевое замечание — правильная оценка $\hat{S}$.
- Дельта-метод удобен для явных функций, таких как логарифмическая, экспоненциальная и обратная.
- Примеры: экспоненциальное и пуассоновское распределения, MLE для λ .
- Практика на Python: вычисление асимптотического доверительного интервала, симуляция покрытия интервала.
- Рекомендации: малые выборки, сильно скошенные распределения, тяжёлые хвосты, параметры на границе области.
- Практические рекомендации: проверка размера выборки, формы распределения, оценка чувствительности интервала, использование корректной формулы для стандартной ошибки, использование более точных методов для параметров у границ.

Практика на Python

Ниже приведены готовые функции, с которыми мы поработаем:

- вычисление асимптотического доверительного интервала (CI) для параметра λ — как для экспоненциального распределения, так и для пуассоновского (через MLE);
- симуляция покрытия интервала, чтобы проверить, насколько часто он действительно содержит истинное значение параметра.

```
import numpy as np
from scipy.stats import norm
np.random.seed(0)
```

```
z95 = norm.ppf(0.975)
```

```
def exp_lambda_ci(data, alpha=0.05):
    n = len(data)
    hat_lambda = 1.0/np.mean(data)
    se = hat_lambda/np.sqrt(n) # plug-in SE via delta method
    lo = hat_lambda - norm.ppf(1-alpha/2)*se
    hi = hat_lambda + norm.ppf(1-alpha/2)*se
    return hat_lambda, lo, hi
```

```
def pois_lambda_ci(data, alpha=0.05):
    n = len(data)
    hat_lambda = np.mean(data)
    se = np.sqrt(hat_lambda/n)
    lo = hat_lambda - norm.ppf(1-alpha/2)*se
    hi = hat_lambda + norm.ppf(1-alpha/2)*se
    return hat_lambda, lo, hi
```

```
# quick example
data_exp = np.random.exponential(scale=1/2.0, size=100) # true lambda=2
print("Exp CI:", exp_lambda_ci(data_exp))
data_pois = np.random.poisson(lam=3.0, size=100)
print("Pois CI:", pois_lambda_ci(data_pois))

# coverage simulation for exp example
def coverage_exp(lambda_true=2.0, n=50, B=2000, alpha=0.05, seed=1):
    rng = np.random.default_rng(seed)
    covered = 0
    for _ in range(B):
        data = rng.exponential(scale=1/lambda_true, size=n)
        hat, lo, hi = exp_lambda_ci(data, alpha=alpha)
        if lo <= lambda_true <= hi:
            covered += 1
    return covered / B

print("Empirical coverage (exp, n=50):", coverage_exp(2.0, n=50, B=2000))
```

Интерпретация следующая: при больших n эмпирическое покрытие $\approx$ заявленное $1-\alpha$; при небольших n (особенно для сильно скошенных распределений) — наблюдаются заметные отклонения.

Урок 7. Проверка гипотез

Статистика — это не только оценка параметров, но и принятие решений. Мы часто стоим перед вопросом: действительно ли эффект существует, или это просто случайность? От лекарства до рекламной кампании, от A/B-теста до научного исследования — всё упирается в проверку гипотез.

В этом уроке мы разберём, как формализовать «сомнение» в данных: построим схему проверки гипотез, поймём, что такое ошибки I и II рода, почему мощность теста так важна, и что на самом деле означает пресловутое p -value. Постепенно перейдём от классических критериев (z , t , χ^2) к более гибким непараметрическим и перестановочным подходам, чтобы уметь выбирать инструмент под задачу, а не наоборот.

- Проверка гипотез - формальный инструмент, помогающий принимать решения в статистике.
- Цель проверки гипотез: понять, согласуются ли данные с нулевой гипотезой (H_0) или дают веские основания отвергнуть ее.
- Ошибки I и II рода и мощность теста - ключевые понятия в статистике.
- Малое p -value - сигнал, что данные мало совместимы с H_0 .
- Частая ошибка: думать, что p -value - это вероятность того, что H_0 истинна.
- На практике важно решить, что дороже: ошибочно заявить эффект или пропустить реальный сигнал.
- Практический алгоритм проверки гипотез: формулирование гипотез, выбор подходящей статистики, проверка предпосылок, определение распределения статистики под H_0 , задание уровня значимости α , построение критической области, сравнение с критическими значениями или использование p -value.
- Примеры и симуляции на Python: z -тест, t -тест, χ^2 -тест, непараметрические и перестановочные тесты.
- Рекомендации и частые ошибки: четкое фиксирование гипотез и условий теста, проверка предпосылок, оценка мощности теста заранее, следование рекомендациям помогает принимать решения на основе статистики обоснованно.

Примеры и симуляции на Python

Симуляция ошибок I/II рода и мощности

```

import numpy as np
from scipy.stats import norm

def z_test_reject(x, mu0=0.0, sigma=1.0, alpha=0.05):
    n = len(x)
    z = (x.mean() - mu0) / (sigma/np.sqrt(n))
    crit = norm.ppf(1-alpha/2)
    return abs(z) > crit

def estimate_power(mu0=0.0, mu1=0.3, sigma=1.0, n=50, alpha=0.05, B=2000, seed=1):
    rng = np.random.default_rng(seed)
    rejects = 0
    for _ in range(B):
        x = rng.normal(loc=mu1, scale=sigma, size=n)
        if z_test_reject(x, mu0=mu0, sigma=sigma, alpha=alpha):
            rejects += 1
    return rejects / B

print("Estimated power (mu1=0.3, n=50):", estimate_power(mu1=0.3, n=50))

```

t-test (scipy) и сравнение с permutation test

```

import numpy as np
from scipy.stats import ttest_1samp

# одновыборочный t-тест
def t_test_reject(x, mu0=0.0, alpha=0.05):
    t_stat, p_value = ttest_1samp(x, mu0)
    return p_value < alpha

# оценка мощности t-теста
def estimate_power_ttest(mu0=0.0, mu1=0.3, sigma=1.0, n=50, alpha=0.05, B=2000, seed=1):
    rng = np.random.default_rng(seed)
    rejects = 0
    for _ in range(B):
        x = rng.normal(loc=mu1, scale=sigma, size=n)
        if t_test_reject(x, mu0=mu0, alpha=alpha):
            rejects += 1
    return rejects / B

print("Estimated power (mu1=0.3, n=50):", estimate_power_ttest(mu1=0.3, n=50))

# перестановочный тест для сравнения двух выборок
def perm_test_mean(a, b, B=2000, seed=1):
    rng = np.random.default_rng(seed)
    obs = a.mean() - b.mean()
    pooled = np.concatenate([a, b])
    count = 0
    for _ in range(B):
        rng.shuffle(pooled)
        new_a = pooled[:len(a)]
        new_b = pooled[len(a):]
        if abs(new_a.mean() - new_b.mean()) >= abs(obs):
            count += 1
    return count / B

# Пример сравнения
a = np.random.normal(0, 1, 30)
b = np.random.normal(0.3, 1, 30)
print("Permutation p:", perm_test_mean(a, b))

```

χ^2 -тест независимости (contingency table)

```
import numpy as np
from scipy.stats import chi2_contingency

# таблица: строки -- лечение/контроль, столбцы -- успех/неуспех
table = np.array([[30, 70],
                  [20, 80]])
chi2, p, dof, ex = chi2_contingency(table)
print("chi2:", chi2, "p:", p, "expected:", ex)
```

Приблизительное вычисление требуемого n для z-теста

Допустим хотим мощность $(1-\beta)$ для обнаружения эффекта размера $(\delta = (\mu_1 - \mu_0)/\sigma)$:

$$n \approx \left(\frac{z_{1-\alpha/2} + z_{1-\beta}}{\delta} \right)^2.$$

На Python расчёт будет выглядеть следующим образом:

```
from scipy.stats import norm
def required_n_z(alpha=0.05, power=0.8, delta=0.3):
    z_a = norm.ppf(1-alpha/2)
    z_b = norm.ppf(power)
    return ((z_a + z_b) / delta)**2

print("n needed (delta=0.3):", int(np.ceil(required_n_z(delta=0.3))))
```

Урок 8. Проверка гипотез о параметрах в одновыборочной гауссовской модели и биномиальных моделях

В этом уроке мы разберём две базовые, но чрезвычайно важные ситуации: когда у нас одна выборка из нормального распределения (проверка среднего при известной или неизвестной дисперсии) и когда у нас серия испытаний Бернулли (проверка доли). Вы научитесь выбирать корректный критерий (z-, t-, точный биномиальный тест или нормальную аппроксимацию), понимать односторонние и двусторонние формулировки, строить доверительные интервалы и оценивать мощность теста. Там, где классическая теория слабо работает (малые n, асимметрия), обсудим альтернативы (Wilson CI, Clopper-Pearson, bootstrap).

- Урок 8 посвящен проверке гипотез о параметрах в одновыборочной гауссовской модели и биномиальных моделях.
- Рассматриваются две базовые ситуации: проверка среднего при известной или неизвестной дисперсии и проверка доли в серии испытаний Бернулли.
- Обсуждаются корректный критерий (z-, t-, точный биномиальный тест или нормальная аппроксимация), односторонние и двусторонние формулировки, построение доверительных интервалов и оценка мощности теста.
- Обсуждаются альтернативы (Wilson CI, Clopper-Pearson, bootstrap) для случаев, где классическая теория слабо работает (малые n, асимметрия).
- Приведены примеры на Python для одновыборочной гауссовской модели и биномиальной модели.
- Обсуждаются оценка мощности и планирование размера выборки.
- Рекомендации и частые ошибки включают использование точного биномиального теста или Clopper-Pearson/Wilson CI при малых n или p близко к 0 или 1, определение односторонней/двусторонней альтернативы до анализа, сообщение вместе с решением CI и p-value для интерпретации, планирование размера выборки заранее для практических A/B-задач.

Одновыборочная гауссовская модель

Пример на Python

Пусть $n = 25$, $\bar{X} = 5.2$, если σ известно и равно 1.1:

```
import numpy as np
from scipy.stats import norm, t, ttest_1samp

n=25; xbar=5.2; sigma=1.1; mu0=5.0
z = (xbar-mu0)/(sigma/np.sqrt(n))
p_two = 2*(1-norm.cdf(abs(z)))
z, p_two
```

Если σ неизвестно, используем *ttest_{1samp}* или формулу для T.

Биномиальная модель

Точный биномиальный тест

Реализация на Python:

```
from scipy.stats import binomtest
res = binomtest(k, n, p=p0, alternative='two-sided')
res.pvalue
```

Нормальное приближение (Wald)

Реализация на Python:

```
import statsmodels.stats.proportion as smp
smp.proportion_confint(count=k, nobs=n, alpha=0.05, method='wilson')
smp.proportion_confint(count=k, nobs=n, alpha=0.05, method='beta') # Clopper-Pearson
```

Ещё пример:

Пусть $n=40$, $k=7$. Точный тест и Wilson CI покажут, насколько $p_0 = 0.1$ согласуется с данными:

```
import scipy.stats as stats
from statsmodels.stats.proportion import proportion_confint

# данные
n = 40
k = 7
p0 = 0.1
alpha = 0.05

# точный биномиальный тест
res = stats.binomtest(k, n, p0, alternative='two-sided')

# доверительный интервал
ci_low, ci_high = proportion_confint(k, n, alpha=alpha, method='wilson')
```

Оценка мощности и планирование размера выборки

Рекомендации и частые ошибки

ПРИМЕРЫ НА PYTHON

Рассмотрим для Z и t:

```
import numpy as np
from scipy.stats import norm, t, ttest_1samp

# пример: sigma известно
n=50; xbar=125.8; sigma=10
z = (xbar-125)/(sigma/np.sqrt(n))
p_two = 2*(1-norm.cdf(abs(z)))

# пример: sigma неизвестен
data = np.random.normal(loc=0.3, scale=1.2, size=25)
tstat, pval = ttest_1samp(data, popmean=0.0)
```

Биномиальный тест и CI:

```
from scipy.stats import binomtest
import statsmodels.stats.proportion as smp

k, n, p0 = 7, 40, 0.1
res = binomtest(k, n, p=p0, alternative='two-sided')
wilson_lo, wilson_hi = smp.proportion_confint(count=k, nobs=n, alpha=0.05, method='wilson')
clopp_lo, clopp_hi = smp.proportion_confint(count=k, nobs=n, alpha=0.05, method='beta') # Clopper-Pearson
res.pvalue, (wilson_lo, wilson_hi), (clopp_lo, clopp_hi)
```

Урок 9. Критерии согласия — критерий Пирсона

В этом уроке мы разберём, как понять, насколько наши данные согласуются с теоретической моделью. Представьте: у вас есть наблюдаемые частоты событий, и вы хотите проверить, «сидят ли» они на той кривой, которую предсказывает теория. Критерий Пирсона χ^2 — один из классических инструментов для такого анализа. Мы научимся не только вычислять статистику χ^2 , но и понимать, что она означает, как правильно группировать данные, рассчитывать ожидаемые частоты и проверять ограничения метода. Кроме того, обсудим альтернативы и корректные приёмы для нестандартных случаев.

- Критерий Пирсона χ^2 используется для проверки согласия наблюдаемых и ожидаемых частот.
- Основная идея: стандартизация квадратичной разности, чтобы получить сумму, имеющую асимптотическое χ^2 -распределение.
- Два типичных сценария применения: Goodness-of-fit и тест независимости в таблице сопряжённости.
- Выбор бинов влияет на корректность теста и интерпретацию результатов.
- Степени свободы определяют критические значения и должны учитываться при интерпретации χ^2 -статистики.
- Ограничения и альтернативы: малые ожидаемые частоты, малые выборки, использование альтернативных методов.
- Примеры на Python: Goodness-of-fit и contingency table.
- Рекомендации и частые ошибки: интерпретация результатов, проверка предпосылок, визуализация данных.
- Следование правилам помогает избежать типичных ошибок и делать корректные выводы.

Ограничения и альтернативы

Goodness-of-fit — пример: тест на соответствие распределению Пуассона

Сгенерируем данные из Пуассона и проверим, подходят ли они под Пуассон с оценённым λ (учтём df).

```
import numpy as np
from scipy.stats import chisquare, poisson
import math

np.random.seed(0)
n = 300
lam_true = 2.3
data = np.random.poisson(lam_true, size=n)

# группируем по значениям 0,1,...,max и объединяем хвост
max_k = data.max()
bins = list(range(0, max_k)) + [max_k+1] # [0],[1],...,[max_k, inf)

# наблюдаемые частоты O_i для значений 0..max_k-1 и остаток
values, counts = np.unique(data, return_counts=True)

# оценка lambda_hat = sample mean
lam_hat = data.mean()
probs = [poisson.pmf(k, lam_hat) for k in range(0, max_k)]
prob_tail = 1 - sum(probs)
probs.append(prob_tail)

E = np.array([p * n for p in probs])
O = np.array([counts[k] if k in values else 0 for k in range(0, max_k)] +
             [sum(counts[values>=max_k])])

# degrees of freedom: k_bins - 1 - m; here m=1 (оценивали lambda)
k_bins = len(O)
df = k_bins - 1 - 1

chi2_stat = ((O - E)**2 / E).sum()
from scipy.stats import chi2
pval = 1 - chi2.cdf(chi2_stat, df)
print("O:", O)
print("E:", np.round(E,2))
print("chi2 stat:", chi2_stat, "df:", df, "p-value:", pval)
```

Contingency table — тест независимости

```
import numpy as np
from scipy.stats import chi2_contingency

# пример 3x2 таблицы
table = np.array([[30, 20],
                  [10, 40],
                  [25, 15]])
chi2, p, dof, expected = chi2_contingency(table, correction=False) # Yates для 2x2
print("chi2:", chi2, "p:", p, "dof:", dof)
print("expected:\n", expected)
```

Монте-Карло p-value для случая малых ожидаемых частот

Если некоторые E_i малы, получим эмуляцию распределения χ^2 под H_0 :

```
import numpy as np

def monte_carlo_chi2_pvalue(observed_counts, expected_probs, B=5000, seed=0):
    rng = np.random.default_rng(seed)
    n = observed_counts.sum()
    obs_stat = (((observed_counts - expected_probs*n)**2) / (expected_probs*n)).sum()
    stats = np.empty(B)
    for i in range(B):
        samp = rng.multinomial(n, expected_probs)
        stats[i] = (((samp - expected_probs*n)**2) / (expected_probs*n)).sum()
    return (stats >= obs_stat).mean(), obs_stat

# пример
expected_probs = np.array(probs) # из примера с пуассоновским биннингом
p_mc, obs_stat = monte_carlo_chi2_pvalue(O, expected_probs, B=5000)
print("Monte Carlo p-value:", p_mc, "obs stat:", obs_stat)
```

Урок 10. Критерий Пирсона для проверки параметрических гипотез

В этом уроке мы разберём, как применять χ^2 -критерий согласия, когда наша нулевая гипотеза задаёт семейство распределений $F(\theta)$ с неизвестными параметрами. Вы научитесь не только вычислять статистику и интерпретировать p-value, но и понимать тонкости: как корректировать число степеней свободы, когда можно доверять асимптотическим результатам, а когда стоит переходить к Monte-Carlo или точным методам. Важно, чтобы проверка гипотез была не механическим подсчётом, а осмысленным анализом — мы хотим видеть, где модель «тянет» данные, а где подводит.

- Критерий Пирсона для проверки параметрических гипотез.
- Сравнение наблюдаемых и ожидаемых частот в каждой категории/бине.
- Расхождение между ними указывает на возможное несоответствие гипотезы и данных.
- Асимптотическое распределение и поправка на оценки параметров важны для больших n .
- Алгоритм применения критерия: задать H_0 , определить I , выбрать разбиение, вычислить ожидаемые частоты, построить статистику χ^2 , найти p-value.
- Практика на Python: проверка Пуассоновского распределения с неизвестным λ .
- Рекомендации и частые ошибки: поправка на число степеней свободы, использование MLE, ожидаемые частоты и подогнанная модель, малые ожидаемые частоты.
- Практические советы по биннингу и выбору числа бинов: избегать слишком большого числа бинов, большинство $E_i \geq 5$, учитывать корректировку df.
- Альтернативные подходы: Likelihood-ratio (G-test), Permutation test и bootstrap.

Практика на Python

Проверка Пуассона с неизвестным (λ)

```
import numpy as np
from scipy.stats import poisson, chi2
np.random.seed(0)

def pearson_chi2_goodness(data, pmf_func, params, max_bin=None):
    # pmf_func(k, *params) -> P(X=k)
    n = len(data)
    if max_bin is None:
        max_bin = data.max()
    # собираем наблюдения по квалам 0..max_bin-1 и хвост >= max_bin
    obs_counts = np.array([np.sum(data==k) for k in range(0, max_bin)] + [np.sum(data>=max_bin)])
```

```

probs = np.array([pmf_func(k, *params) for k in range(0, max_bin)])
probs = np.append(probs, 1.0 - probs.sum())
exp_counts = probs * n
# объединяем хвосты, если нужны (здесь уже сделано)
chi2_stat = ((obs_counts - exp_counts)**2 / exp_counts).sum()
return chi2_stat, obs_counts, exp_counts

# симуляция
n = 300
lambda_true = 2.3
data = np.random.poisson(lambda_true, size=n)
# оценка lambda (MLE = sample mean)
lambda_hat = data.mean()
# выберем max_bin так, чтобы хвост не был слишком мал
chi2_stat, O, E = pearson_chi2_goodness(data, poisson.pmf, (lambda_hat,), max_bin=6)
k_bins = len(O)
l = 1 # число оценённых параметров (lambda)
df = k_bins - 1 - l
pval = 1 - chi2.cdf(chi2_stat, df)
print("chi2:", chi2_stat, "df:", df, "p-value:", pval)
print("O:", O)
print("E:", np.round(E,2))

```

Monte-Carlo p-value (когда асимптотика ненадёжна)

Если некоторые E_i малы или n невелико — имитируем распределение статистики под H_0 условно на $(\hat{\theta})$:

```
import numpy as np
```

```

def monte_carlo_pvalue(data, pmf_sampler, params_hat, max_bin, B=5000, seed=1):
    rng = np.random.default_rng(seed)
    n = len(data)
    # observed statistic
    chi2_obs, O_obs, E_obs = pearson_chi2_goodness(data, pmf_sampler.pmf, params_hat,
max_bin)
    stats = np.empty(B)
    for b in range(B):
        sim = pmf_sampler.rvs(size=n, **params_hat) if hasattr(pmf_sampler, 'rvs') else None
        # pmf_sampler here can be scipy frozen distribution; params_hat as callable dictionary
        # for simplicity, assume poisson: rng.poisson(lam=params_hat['mu'], size=n)
        # adjust according to distribution in real code
        # --- here a simple example for Poisson:
        sim = rng.poisson(lam=params_hat['lam'], size=n)
        chi2_sim, _, _ = pearson_chi2_goodness(sim, poisson.pmf, (params_hat['lam'],), max_bin)
        stats[b] = chi2_sim
    p_mc = (stats >= chi2_obs).mean()
    return p_mc, chi2_obs

```

```

# пример использования (для Poisson)
params_hat = {'lam': lambda_hat}
p_mc, chi2_obs = monte_carlo_pvalue(data, poisson, params_hat, max_bin=6, B=2000, seed=2)
print("Monte-Carlo p-value:", p_mc, "chi2_obs:", chi2_obs)

```

Урок 11. Однородность нормальных выборок

Когда мы сравниваем две группы, часто хочется понять не только различаются ли их средние, но и насколько схожи их разбросы. Равенство дисперсий — важная предпосылка для многих тестов (например, t-теста для независимых выборок). Этот юнит посвящён

тому, как проверять, что две нормальные выборки «однородны» по дисперсии: классический F-критерий, доверительные интервалы для отношения дисперсий и более робастные альтернативы. Мы разберём, когда стандартные методы работают хорошо, а когда лучше подключать Levene, Bartlett или Brown–Forsythe.

- Задача: проверить гипотезу о равенстве дисперсий двух нормальных выборок.
- Несмещенные оценки дисперсий: S_X^2 и S_Y^2 .
- Статистика $F = S_X^2 / S_Y^2$.
- Критическая область и p-value для правостороннего, левостороннего и двустороннего тестов.
- Доверительный интервал для отношения дисперсий: $(S_X^2 / S_Y^2 \cdot 1/F_{1-\alpha/2}, n-1, m-1, S_X^2 / S_Y^2 \cdot 1/F_{\alpha/2}, n-1, m-1)$.
- Связь с t-тестом: классическое "pooled" t-test и Welch t-test.
- Чувствительность и робастные альтернативы: Levene, Brown-Forsythe и Bartlett тесты.
- Практический совет: использовать Welch t-test вместо проверки дисперсий.
- Рекомендации и частые ошибки: проверка нормальности данных, выбор метода проверки однородности, учет аномалий и визуализации результатов.

Примеры на Python

КЛАССИЧЕСКИЙ F-ТЕСТ И CI ДЛЯ ОТНОШЕНИЕ ДИСПЕРСИЙ

```
import numpy as np
from scipy.stats import f

np.random.seed(0)
n, m = 20, 18
# сгенерируем две нормальные выборки
X = np.random.normal(loc=0.0, scale=1.5, size=n)
Y = np.random.normal(loc=0.0, scale=1.0, size=m)

Sx2 = X.var(ddof=1)
Sy2 = Y.var(ddof=1)
F_stat = Sx2 / Sy2
df1, df2 = n-1, m-1

# двусторонний p-value: аккуратно
p_one_tail = 1 - f.cdf(F_stat, df1, df2)
p_two_tail = 2 * min(p_one_tail, f.cdf(F_stat, df1, df2))

print("Sx2, Sy2:", Sx2, Sy2)
print("F stat:", F_stat, "p(two-tailed):", p_two_tail)

# доверительный интервал для ratio sigma1^2 / sigma2^2
alpha = 0.05
lower = F_stat / f.ppf(1 - alpha/2, df1, df2)
upper = F_stat / f.ppf(alpha/2, df1, df2)
print("CI for sigma1^2/sigma2^2:", (lower, upper))
```

LEVENE И BARTLETT (РОБАСТНОСТЬ)

```
from scipy.stats import levene, bartlett

lev_stat, lev_p = levene(X, Y, center='median') # center='mean' or 'median'
bart_stat, bart_p = bartlett(X, Y)

print("Levene p:", lev_p)
print("Bartlett p:", bart_p)
```

Урок 12. Критерии, основанные на доверительных интервалах

Доверительные интервалы — это не только способ оценить параметр, но и простой инструмент для проверки гипотез. В этом юните мы разберём, как использовать интервалы для принятия статистических решений: входит ли предполагаемое значение параметра в интервал — значит, гипотеза не отвергается; не входит — отвергаем. Вы увидите, как уровень значимости связан с уровнем доверия, и научитесь формулировать критерии проверки гипотез прямо через доверительные интервалы.

- Доверительные интервалы используются для оценки параметров и проверки гипотез.
- Интервалы могут быть двусторонними или односторонними, в зависимости от задачи.
- Интерпретация через доверительные интервалы делает статистический вывод наглядным.
- Однако, подход через доверительные интервалы требует аккуратности и учета некоторых особенностей.
- Примеры на Python демонстрируют использование доверительных интервалов для различных задач.

В итоге, подход через доверительные интервалы удобен и нагляден, но требует аккуратности: несогласованность методов, асимметрия или множественные проверки могут привести к неправильным выводам.

Примеры на Python

Реализуем рассмотренные приемы на Python:

```
import numpy as np
from scipy.stats import norm, t
import statsmodels.stats.proportion as smp
from scipy.stats import bootstrap
```

```
np.random.seed(1)
```

```
# 1) z-interval (sigma known)
```

```
n = 50
```

```
mu_true = 1.0
```

```
sigma = 1.2
```

```
x = np.random.normal(mu_true, sigma, size=n)
```

```
xbar = x.mean()
```

```
alpha = 0.05
```

```
z = norm.ppf(1-alpha/2)
```

```
ci_z = (xbar - z * sigma/np.sqrt(n), xbar + z * sigma/np.sqrt(n))
```

```
print("z-CI:", ci_z)
```

```
print("Reject H0 mu0=0?" , not (0>=ci_z[0] and 0<=ci_z[1]))
```

```
# 2) t-interval (sigma unknown)
```

```
s = x.std(ddof=1)
```

```
tq = t.ppf(1-alpha/2, df=n-1)
```

```
ci_t = (xbar - tq * s/np.sqrt(n), xbar + tq * s/np.sqrt(n))
```

```
print("t-CI:", ci_t)
```

```
# 3) proportion: exact binomial test via inversion of CI
```

```
k = 7; N = 40
```

```
wilson = smp.proportion_confint(count=k, nobs=N, alpha=alpha, method='wilson')
```

```
cp = smp.proportion_confint(count=k, nobs=N, alpha=alpha, method='beta') # Clopper-Pearson
```

```
print("Wilson CI:", wilson)
```

```
print("Clopper-Pearson CI:", cp)
```

```
# test H0: p=0.1 by checking inclusion
```

```

p0 = 0.1
print("Reject H0 via Wilson?", not (wilson[0] <= p0 <= wilson[1]))

# 4) bootstrap percentile CI for mean (simple)
def mean_stat(data, axis):
    return np.mean(data, axis=axis)

res = bootstrap(x, np.mean, vectorized=False, paired=False, confidence_level=0.95,
n_resamples=5000, method='percentile', random_state=2)
print("Bootstrap percentile CI for mean:", res.confidence_interval)

# 5) difference of means (Welch CI) and test via CI
a = np.random.normal(0.5, 1.0, size=30)
b = np.random.normal(0.0, 1.3, size=40)
diff = a.mean() - b.mean()
se = np.sqrt(a.var(ddof=1)/len(a) + b.var(ddof=1)/len(b))
# Welch df
num = (a.var(ddof=1)/len(a) + b.var(ddof=1)/len(b))**2
den = (a.var(ddof=1)/len(a))**2/(len(a)-1) + (b.var(ddof=1)/len(b))**2/(len(b)-1)
df_w = num/den
tq_w = t.ppf(1-alpha/2, df=df_w)
ci_welch = (diff - tq_w*se, diff + tq_w*se)
print("Welch CI for mean diff:", ci_welch)
print("Reject H0: diff=0?", not (ci_welch[0] <= 0 <= ci_welch[1]))

```

Заключение по модулю

Поздравляем с прохождением модуля! Теперь вы знаете, как связать теорию статистики с практикой анализа данных и планирования экспериментов: какие критерии и где применять, чем они ограничены и как сделать выводы, отражающие реальную структуру данных.

Теперь мы умеем:

- Строить и интерпретировать доверительные и предсказательные интервалы, включая классические (z, t, χ^2) и робастные (bootstrap, permutation) варианты.
- Проверять гипотезы о параметрах и долях, формулировать H_0 / H_1 , выбирать статистику, вычислять p-value и оценивать мощность теста.
- Применять классические тесты: z , t (pooled и Welch), χ^2 (goodness-of-fit и независимость), F , ANOVA, биномиальные и непараметрические (Wilcoxon, Kruskal–Wallis).
- Работать с ключевыми распределениями: χ^2 , t , F , понимать их происхождение и области применения.
- Оценивать и учитывать ограничения методов: нормальность, равенство дисперсий, малые n , асимметрию, а также применять альтернативы (Levene, Brown–Forsythe, bootstrap, permutation).
- Планировать размер выборки и мощность теста для корректного эксперимента.
- Проводить анализ данных в Python: строить интервалы, выполнять тесты, симулировать покрытие и мощность, визуализировать результаты для интерпретации.

На что обратить особое внимание:

- Формулируйте H_0 и H_1 чётко и заранее.
- Проверяйте предпосылки: нормальность, гомоскедастичность, независимость.
- Для малых выборок или при асимметрии предпочтительнее использовать bootstrap или permutation.
- Welch t-test безопаснее при сравнении средних с неравными дисперсиями.
- Доверительные интервалы вместе с p-value дают количественную и практическую интерпретацию результатов.

- При множественных тестах контролируйте FWER/FDR (Bonferroni, Holm, Benjamini–Hochberg).
- Визуализация (hist, boxplot, QQ-plot, bar/OvsE) часто проясняет данные лучше, чем десятки p-value.

Следование этим правилам помогает строить корректные выводы, понимать, когда классические методы подходят, а когда лучше подключать робастные альтернативы. Теперь можно уверенно применять статистику в анализе данных, планировать эксперименты и визуализировать результаты так, чтобы они были понятны и практичны.

В следующем модуле мы перейдём к исследованию взаимосвязей между переменными: изучим корреляционный анализ и познакомимся с регрессионным анализом, включая линейную и множественную регрессию, чтобы прогнозировать значения на основе известных факторов.

Домашнее задание №1

Теоретическое введение

В предыдущих лекциях были изучены критерии для проверки однородности (или неоднородности) двух выборок при предположении нормальности. Теперь обобщим задачу на случай трёх и более выборок. Пусть имеется l выборок (групп), полученных независимо друг от друга из l нормально распределённых генеральных совокупностей, которые могут иметь различные средние $a_1, \dots, a_l$ и одинаковую дисперсию σ^2 . Нас интересует проверка гипотезы

$$H_0 : a_1 = a_2 = \dots = a_l \quad \text{против} \quad H_1 : \exists i \neq j : a_i \neq a_j.$$

Эта задача называется одномерным (однофакторным) дисперсионным анализом (one-way ANOVA). Физический смысл: у нас есть фактор с l уровнями (например, разные добавки в рационе, разные методики обучения и т.п.), и мы хотим узнать, влияет ли уровень фактора на среднее значение отклика.

Обозначения. Пусть x_{ik} — i -й наблюдаемый элемент в k -й группе, где $k = 1, \dots, l, i = 1, \dots, n_k$. Обозначим

$$X_k = \frac{1}{n_k} \sum_{i=1}^{n_k} x_{ik} \quad \text{— выборочное среднее группы } k, \quad n = \sum_{k=1}^l n_k \quad \text{— общее число наблюдений,}$$

$$X = \frac{1}{n} \sum_{k=1}^l \sum_{i=1}^{n_k} x_{ik} \quad \text{— общее (пулевое) среднее.}$$

Суммы квадратов и разложение (основное тождество ANOVA). Общая сумма квадратов отклонений от общего среднего:

$$Q = \sum_{k=1}^l \sum_{i=1}^{n_k} (x_{ik} - X)^2.$$

Её можно разложить на вклад между группами и внутри групп:

$$Q = \underbrace{\sum_{k=1}^l n_k (X_k - X)^2}_{Q_1 \text{ (между группами)}} + \underbrace{\sum_{k=1}^l \sum_{i=1}^{n_k} (x_{ik} - X_k)^2}_{Q_2 \text{ (внутри групп)}}.$$

При верной гипотезе H_0 имеют место следующие распределения:

$$\frac{Q_1}{\sigma^2} \sim \chi_{l-1}^2, \quad \frac{Q_2}{\sigma^2} \sim \chi_{n-l}^2,$$

и эти две статистики независимы. Отсюда несмещённые оценки дисперсии

$$S_1^2 = \frac{Q_1}{l-1}, \quad S_2^2 = \frac{Q_2}{n-l},$$

где S_1^2 оценивает вариацию средних между группами, а S_2^2 — вариацию внутри групп (шум). Статистика отношения этих оценок

$$F = \frac{S_1^2}{S_2^2} = \frac{Q_1/(l-1)}{Q_2/(n-l)}$$

при H_0 распределена по закону Фишера с параметрами $(l-1, n-l)$.

Критерий: при уровне значимости α отклоняем H_0 , если

$$F \geq F_{1-\alpha; l-1, n-l}.$$

Альтернативный способ — вычислить p -value как $p = 1 - F_F(F_{\text{obs}})$ и сравнить с α .

Практические замечания и проверки допущений.

- Допущения ANOVA:
 - независимость наблюдений (важнейшее);
 - нормальность распределения остатков в каждой группе (проверяется графиками — Q-Q plot — или тестом Шапиро-Уилка);
 - однородность дисперсий (гомоскедастичность) между группами (проверяется тестами Левена или Бартлетта).
- Если дисперсии не равны: классическая ANOVA может давать искажённые результаты при сильной неоднородности дисперсий и особенно при несбалансированных выборках. В этом случае применяют:
 - Welch's ANOVA (не требует равенства дисперсий);
 - тест Брауна–Форсайта;
 - в качестве post-hoc — коррективы для неравных дисперсий (Games-Howell).

- Пост-хоc сравнения. Если H_0 отвергнута, обычно выполняют попарные сравнения средних с контролем уровня ошибки множественных сравнений:
 - Tukey HSD (подходит при равных дисперсиях и одинаковых/приблизительно одинаковых n_k);
 - Bonferroni, Holm — более консервативные поправки;
 - при неравных дисперсиях — Games-Howell.
- Эффект размера. Помимо значимости, полезно оценивать величину эффекта:
 - $\eta^2 = \frac{Q_1}{Q}$ — наивная доля дисперсии, объяснённой фактором;
 - скорректированная оценка (omega squared) $\omega^2 = \frac{Q_1 - (l - 1)S_2^2}{Q + (S_2^2)}$
(существуют точные формулы; важно указывать используемую версию).
- ANOVA при несбалансированных выборках. Для разных n_k формулы для сумм квадратов остаются те же, но при интерпретации и в вычислении некоторых типов последовательных (type I/II/III) сумм квадратов важно учитывать модель (фиксированный/случайный эффект) и порядок факторов.
- Практическая схема анализа:
 - визуализация данных (ящики с усами, точки, средние);
 - проверка допущений (Shapiro, Levene/Bartlett, визуальные проверки);
 - классическая однофакторная ANOVA или Welch-ANOVA;
 - при отклонении H_0 : post-hoc тесты и оценка величины эффекта;
 - содержательная интерпретация результатов.

Пример. Имеются данные (в рублях) Федеральной службы государственной статистики о среднедушевых месячных доходах населения в 2008 г. по некоторым областям Центрального, Приволжского и Дальневосточного федеральных округов. Данные представлены в таблице.

Можно ли считать, что средние значения среднедушевых месячных доходов населения одинаковы во всех трех округах?

Решение. Фактором, т. е. переменной, которая может оказывать влияние на измеряемую величину (среднедушевой доход), является федеральный округ. Фактор в данном случае имеет три уровня: 1 — «Центральный федеральный округ», 2 — «Приволжский федеральный округ», 3 — «Дальневосточный федеральный округ». Имеющиеся данные представляются тремя выборками объема $n_1 = 11$, объема $n_2 = 8$ и объема $n_3 = 6$.

Можем считать, что рассматриваемые выборки соответствуют нормальному (гауссовскому) распределению, проверим гипотезу об однородности: $H_0 : (a_1 = a_2 = a_3)$ против альтернативы: $H_1 : \exists a_i \neq a_j, i \neq j$.

Для данного случая $l = 3$.

округ	Среднедушевой доход по различным областям	Сумма вариант	Число областей	Выборочные средние
Центральный ФО	10305, 8354, 9413, 19776, 9815, 11311, 11253, 10856, 11389	122111	11	11101,0
Приволжский ФО	10112, 7843, 9581, 16119, 8594, 14253, 10173, 9756	86431	8	10803,9
Дальневосточный ФО	19063, 19703, 24552, 15705, 10877, 32140	122040	6	20340,0

Общее	330582	25	13223,3
-------	--------	----	---------

$$S_1^2 = 200129591$$

$$S_2^2 = 19038613$$

$$\rho = 10.51$$

Построим критерий с уровнем значимости $\epsilon = 0.05$:

$$\delta(x_1, x_2, x_3) = \begin{pmatrix} H_{0, \rho < F_{0.95, 2.22} = 3.44} \\ H_{1, \rho \geq F_{0.95, 2.22} = 3.44} \end{pmatrix}$$

Таким образом, наблюдаемое значение статистики попадает в критическую область и гипотеза об однородности отвергается на уровне значимости 0,05, поэтому нельзя считать, что среднее значение показателей среднедушевых месячных доходов населения одинаково во всех трех округах.

Домашнее задание

Вам предлагается ряд ситуаций, где нужно ответить на поставленный вопрос, используя некоторую статистику из рассмотренных в модуле.

Ваша задача — выполнить задания в соответствии с критериями оценивания. Для оформления вам предлагается шаблон — `ipynb`-файл, который нужно скопировать себе и в него уже вставить требуемый код.

Ссылка на шаблон:

https://colab.research.google.com/drive/1jjNqSdJABWW1g4Yw29O2vIFS_aYQhI_5#scrollTo=11a48b67

В качестве ответа прикрепите файл в формате `.ipynb` со своим кодом.

Критерии оценивания

Задание 1. Сопоставление статистик

Вариант оценки	Балл	Пояснение
Выполнено	2	Верно указаны все применяемые статистики для каждой ситуации; даны корректные краткие обоснования (1–2
Нуждается в доработке	1	Основные тесты подобраны верно, но отдельные объяснения неточны или отсутствуют; возможны 1–2 ошибки в выборе
Не выполнено	0	Большинство ответов неверны или отсутствует логика выбора статистических процедур

Максимум: 2 балла.

Задание 2. Тест для дозатора (z-test)

Вариант оценки	Балл	Пояснение
Выполнено	2	Корректно сформулированы гипотезы и выбран z-тест; представлены расчёты (z_{obs} , p-value, доверительный интервал), верно сделан практический вывод о необходимости
Нуждается в доработке	1	Тест выбран верно, но вычисления частично неверны или неполны (например, не указан CI или p-value), вывод сделан без
Не выполнено	0	Тест выбран некорректно или отсутствуют расчёты и вывод.

Максимум: 2 балла.

Задание 3. Сравнение прочности материалов

Вариант оценки	Балл	Пояснение
Выполнено	2	Проведены оба теста на равенство дисперсий (Levene и F); выбран и применён корректный t-тест (pooled или Welch) в зависимости от результатов; рассчитаны p-value и
Нуждается в доработке	1	Основные тесты выполнены, но отсутствует проверка допущений, неверно выбран тип t-теста, либо не приведён
Не выполнено	0	Выбор теста и расчёты некорректны или отсутствуют.

Максимум: 2 балла.

Задание 4. Парные измерения — новая vs старая схема

Вариант оценки	Балл	Пояснение
Выполнено	2	Проверена нормальность разностей (Shapiro–Wilk); выбран и корректно выполнен парный t-test или Wilcoxon; рассчитаны p-value, доверительный интервал и эффект (Cohen's d); дан
Нуждается в доработке	1	Проверка нормальности или эффект рассчитаны частично или неверно; тест выполнен, но без интерпретации или
Не выполнено	0	Тест выбран неверно или отсутствует полный анализ.

Максимум: 2 балла.

Задание 5. До/после — медицинский пример (paired)

Вариант оценки	Балл	Пояснение
Выполнено	2	Проверена нормальность разностей; выполнен корректный парный t-test или Wilcoxon (при нарушении нормальности); даны
Нуждается в доработке	1	Основные расчёты присутствуют, но нет проверки нормальности или неполная интерпретация результата.
Не выполнено	0	Тест выбран неверно или отсутствует анализ.

Максимум: 2 балла.

Задание 6. Welch и Games–Howell (симуляция с неравными дисперсиями)

Вариант оценки	Балл	Пояснение
Выполнено	2	Проведена корректная симуляция данных с неравными дисперсиями; выполнены классическая ANOVA и Welch ANOVA; проведён post-hoc тест (Games–Howell); дан содержательный
Нуждается в доработке	1	Симуляция и тесты выполнены, но нет post-hoc анализа или интерпретация ограничена; возможны незначительные ошибки
Не выполнено	0	Симуляция или анализ не выполнены, либо выводы противоречат результатам тестов

Максимум: 2 балла.

Задание 7. ANOVA на реальных данных

Вариант оценки	Балл	Пояснение
Выполнено	2	Верно указана методология и обоснование (проверки допущений: Shapiro по группам и Levene); получена ANOVA-таблица (SS, df, MS, F, p-value); выполнен корректный post-hoc тест (Tukey HSD или Games–Howell); вычислен и интерпретирован размер эффекта (η^2)
Нуждается в доработке	1	Основные расчёты выполнены, но отсутствует post-hoc при необходимости, либо размер эффекта или интерпретация
Не выполнено	0	Ключевые шаги (ANOVA, post-hoc, интерпретация) не выполнены или выполнены с ошибками.

Максимум: 2 балла.

Задание 8. Планирование эксперимента: размер выборки и мощность

Вариант оценки	Балл	Пояснение
----------------	------	-----------

Выполнено	2	Корректно указаны формулы связи мощности, размера эффекта и объёма выборки; выполнена численная оценка η^2 ;
Нуждается в доработке	1	Формулы указаны частично, численная оценка неполна или без пояснений; интерпретация ограничена.
Не выполнено	0	Формулы и расчёты отсутствуют или выполнены некорректно.

Максимум: 2 балла.

МАКСИМУМ 16 БАЛЛОВ ЗА ЗАДАНИЕ.

Домашнее задание №2.

Корреляция и регрессия

Вам предлагается файл [practical_module_dataset.csv](#) для выполнения практической работы по корреляции и регрессии.

Ваша задача — выполнить задания в соответствии с критериями оценивания. Для оформления вам предлагается шаблон — `ipynb`-файл, который нужно скопировать себе и в него уже вставить требуемый код, в нём же описаны и все условия задач.

Ссылка на шаблон:

https://colab.research.google.com/drive/1QqJ9yOcXfFssDlwPAFEv_YDtI2mZ7kZA

В качестве ответа прикрепите файл в формате `.ipynb` со своим кодом.

Общие требования к оформлению

- В ячейках ноутбука оставляйте краткие выводы 1–2 фразы после каждого задания, а также выводите необходимые численные ответы на экран.
- Для воспроизводимости используйте `random_state` / `np.random.seed` там, где это требуется.

Критерии оценивания

Критерий 1. Предварительное исследование данных (EDA)

Вариант оценки	Балл	Пояснение
Выполнено	1	Проведена проверка типов столбцов и наличия пропусков. Построены гистограммы для всех числовых переменных. Краткий вывод о распределениях (симметрия/асимметрия, наличие выбросов).
Не выполнено	0	Одно или более требований не выполнено или выполнено неверно.

Критерий 2. Визуальный анализ `height_cm` vs `weight_kg` и расчёт корреляции Пирсона

Вариант оценки	Балл	Пояснение
Выполнено	2	Построен scatter plot <code>height_cm</code> vs <code>weight_kg</code> . Рассчитан коэффициент Пирсона <code>r</code> и <code>p-value</code> . Дана корректная интерпретация силы и направления связи и комментарий о значимости.
Частично	1	Построен график или рассчитан <code>r</code> , но отсутствует корректная интерпретация или пропущен <code>p-value</code> .
Не выполнено	0	Ни графика, ни расчёта <code>r</code> нет.

Критерий 3. Расчёт коэффициента Спирмена между tv_hours и iq

Вариант оценки	Балл	Пояснение
Выполнено	1	Рассчитан Spearman r_s и p-value. Дана интерпретация (есть/нет монотонная связь).
Не выполнено	0	Расчёт или интерпретация отсутствуют/ошибочны.

Критерий 4. Простая линейная регрессия weight_kg~height_cm — коэффициенты и CI

Вариант оценки	Балл	Пояснение
Выполнено	2	Найдены intercept и slope (statsmodels/sklearn). Рассчитан 95% CI для наклона. Дана интерпретация наклона и CI.
Частично	1	Коэффициенты найдены, но CI отсутствует или неверно рассчитан/интерпретирован.
Не выполнено	0	Коэффициенты не найдены или расчёты неверны.

Критерий 5. Предсказание и 95% предсказательный интервал для height_cm=175

Вариант оценки	Балл	Пояснение
Выполнено	1	Вычислено точечное предсказание и корректный 95% prediction (obs) interval для нового наблюдения; дана интерпретация интервала.
Не выполнено	0	Интервал/предсказание отсутствуют или неправильно рассчитаны.

Критерий 6. Множественная регрессия score ~ study_hours + iq + tv_hours + age

Вариант оценки	Балл	Пояснение
Выполнено	2	Построена модель, представлены коэффициенты, стандартные ошибки и p-values. Рассчитан и указан R^2 . Даны комментарии о значимости предикторов.
Частично	1	Модель построена, но отсутствует интерпретация p-values или R^2 .
Не выполнено	0	Модель не построена или результаты неверны.

Критерий 7. Train/test split и оценка качества (RMSE)

Вариант оценки	Балл	Пояснение
----------------	------	-----------

Выполнено	1	Данные корректно разделены (70/30, фиксирован random_state). Модель обучена на train и RMSE вычислен на test; указано числовое значение RMSE и краткая интерпретация.
Не выполнено	0	Разделение/обучение/оценка отсутствуют или ошибочны.

Критерий 8. Градиентный спуск для score~study_hours

Вариант оценки	Балл	Пояснение
Выполнено	2	Реализован градиентный спуск (с intercept), указаны параметры (learning rate, число итераций). Полученные коэффициенты сопоставлены с OLS и объяснены возможные расхождения. Код читаемый и корректный.
Частично	1	Реализация есть, но без сравнения с OLS или без intercept; либо параметры подобраны плохо и нет объяснения.
Не выполнено	0	Градиентный спуск не реализован или реализован неверно.

Критерий 9. Бутстрэп 95% CI для Pearson r (height vs weight)

Вариант оценки	Балл	Пояснение
Выполнено	2	Проведено ≥ 1000 бутстрэп-репликаций, корректно вычислены границы 95% CI (перцентильный метод). Приложено краткое пояснение интерпретации.
Частично	1	Бутстрэп выполнен, но слишком мало реплик (< 500) или CI рассчитан неверно.
Не выполнено	0	Бутстрэп отсутствует или ошибка в реализации.

Критерий 10. Сохранение очищенного набора и анализ остатков

Вариант оценки	Балл	Пояснение
Выполнено	2	Построена гистограмма/diagnostic plot остатков для модели из задания 6, дана краткая интерпретация (форма распределения, асимметрия, выбросы).
Не выполнено	0	График остатков отсутствует или неверно построен.

МАКСИМУМ 16 БАЛЛОВ ЗА ЗАДАНИЕ.

Домашнее задание "Корреляционный и регрессионный анализ"

Инструкции:

- Скачайте файл `practical\_module\_dataset.csv` и заполните ячейки с заданиями кодом и выводами.
- В каждой задаче требуется:
 - короткая формулировка метода и обоснование,
 - код/расчёты,
 - результаты (коэффициенты, статистики, p-value, CI, R^2),
 - промежуточный вывод.
- Используйте `numpy`, `pandas`, `scipy`, `statsmodels` или `scikit-learn` по желанию. При необходимости предобработки данных (очистка, удаление пропусков) укажите свои действия.
- Для воспроизводимости задан `np.random.seed` - не меняйте его!

Формат сдачи: `ipynb` с заполненными ячейками, кодом и выводами.

ячейка с импортами - дополняйте по своему желанию!

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import statsmodels.api as sm
from scipy import stats
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
```

```
np.random.seed(28)
```

```
df = pd.read_csv('practical_module_dataset.csv')
df.shape
df.head()
```

Задание 1.

Выполните первичный анализ данных (EDA):

1. посмотрите на `describe()` для всех переменных,
2. проверьте наличие пропусков и типы данных через `info()`,
3. запишите наблюдения в текстовой ячейке.

Ваш код и расчёты

Ваш код и расчёты

<Ваши ответы, рассуждения и выводы>

Задание 2.

1. Постройте scatter plot по признакам `height\_cm` и `weight\_kg`.
2. Посчитайте коэффициент Пирсона между `height\_cm` и `weight\_kg` и выведите его на экран.
3. Интерпретируйте результат и запишите в виде мини-вывода.

Ваш код и расчёты

Ваш код и расчёты

<Ваши ответы, рассуждения и выводы>

Задание 3.

1. Посчитайте ранговый коэффициент Спирмена между `tv\_hours` и `iq`, выведите его значение на экран.
2. Оцените, есть ли связь? Запишите наблюдения в виде короткого вывода.

Ваш код и расчёты

Ваш код и расчёты

<Ваши ответы, рассуждения и выводы>

Задание 4.

1. Постройте простую линейную регрессию ``weight_kg ~ height_cm``.
2. Найдите и выведите оценки коэффициентов (intercept и slope).
3. Рассчитайте и выведите 95% доверительный интервал для наклона.
4. Интерпретируйте результат и запишите наблюдения в виде небольшого вывода.

Ваш код и расчёты

Ваш код и расчёты

<Ваши ответы, рассуждения и выводы>

Задание 5.

Для ``height_cm = 175``:

1. вычислите предсказание веса,
2. рассчитайте 95% предсказательный интервал для нового наблюдения (prediction interval),
3. запишите небольшой вывод.

Ваш код и расчёты

Ваш код и расчёты

<Ваши ответы, рассуждения и выводы>

Задание 6.

1. Постройте множественную регрессию ``score ~ study_hours + iq + tv_hours + age``.
2. Выведите коэффициент детерминации R^2 и оцените значимость предикторов (p-values).
3. Запишите наблюдения в вывод.

Ваш код и расчёты

Ваш код и расчёты

<Ваши ответы, рассуждения и выводы>

Задание 7.

1. Разделите данные на train/test (70/30), используйте ``random_state=42``.
2. Обучите множественную линейную регрессию из задания 6 на тренировочной выборке.
3. И оцените RMSE на тестовой выборке - выведите её на экран.
4. Насколько модель хорошо предсказывает целевую переменную?

Ваш код и расчёты

Ваш код и расчёты

<Ваши ответы, рассуждения и выводы>

Задание 8.

1. Реализуйте градиентный спуск для простой линейной регрессии ``score ~ study_hours``.
2. Сравните полученные коэффициенты со ``statsmodels``.

PS: выведите коэффициенты в обоих случаях на экран.

Ваш код и расчёты

Ваш код и расчёты

<Ваши ответы, рассуждения и выводы>

Задание 9.

Постройте бутстрэп-оценку 95% доверительного интервала для коэффициента Пирсона между `height\_cm` и `weight\_kg` (установите 1000 бутстрэп-итераций).

Выведите границы ДИ на экран.

Ваш код и расчёты

Ваш код и расчёты

<Ваши ответы, рассуждения и выводы>

Задание 10.

Постройте гистограмму остатков (residuals) для модели из задания 6.

Ваш код и расчёты

Ваш код и расчёты

<Ваши ответы, рассуждения и выводы>

Удачи! Сохраните ноутбук, перезапустите все ячейки от начала до конца и прикрепите файл как ответ.

Корреляция и регрессионный анализ

Урок 1. Введение

Когда мы анализируем большой набор данных, часто встают важные вопросы о том, каким образом данные связаны между собой, как изменение одного параметра влияет на другой и возможно ли предсказать значение неизвестного параметра по известным.

Конечно, некоторые причинно-следственные связи очевидны, исходя из нашего жизненного опыта (например, днем всегда теплее, чем ночью). Однако в большинстве случаев определить степень взаимного влияния данных друг на друга на интуитивном уровне невозможно.

Если пренебречь хотя бы простейшим анализом взаимосвязи данных, то можно столкнуться с рядом существенных проблем. Например, если параметр А увеличивается примерно в 3 раза при увеличении параметра В в 2 раза в 95% случаев, то с большой долей вероятности у нас пропадает потребность строить сложную модель зависимости параметра А от параметров С, D и т. д. То есть фактически мы отбрасываем малозначимые признаки, тем самым упрощаем модель для понимания и вычислений.

Также исследование взаимосвязи данных позволит нам ответить на вопрос о том, как изменится параметр А, если мы изменим параметр В не в 2, а в 3 раза, тем самым установив численные отношения между параметрами А и В на всем множестве их возможных значений и изменений.

Данный модуль посвящен таким методам обработки данных, как корреляционный и регрессионный анализ — двум наиболее часто используемым методам для исследования взаимосвязи между количественными переменными.

Основные темы модуля:

- Корреляционный анализ. Здесь мы изучим методы, которые позволяют определить, существует ли взаимосвязь между тестируемыми переменными. В частности, будут рассмотрены корреляции Пирсона и Спирмена.
- Регрессионный анализ. Здесь мы рассмотрим методы, которые позволяют оценить значение неизвестной переменной, используя знание известных переменных. В частности, мы рассмотрим линейную и множественную регрессию.

Итак, этот модуль научит вас определять наличие взаимосвязи между параметрами, тип этой взаимосвязи и количественно оценивать значение неизвестной величины по известным величинам.

Урок 2. Корреляционный анализ

В этом уроке мы научимся количественно описывать и проверять взаимосвязь между двумя переменными. Сначала рассмотрим интуитивное объяснение, затем формулы и критерии значимости, познакомимся с двумя основными коэффициентами (Пирсона и Спирмена), обсудим их ограничения и практические приёмы (визуализация, влияние выбросов, доверительные интервалы).

- Корреляционный анализ позволяет количественно описывать и проверять взаимосвязь между двумя числовыми признаками.
- Корреляция может быть линейной или монотонной, и ее значение варьируется от -1 до 1.

- Коэффициент корреляции Пирсона и Спирмена являются основными коэффициентами для оценки линейной и монотонной зависимости соответственно.
- Тест значимости корреляции используется для проверки ее статистической значимости.
- Доверительный интервал для истинного коэффициента корреляции позволяет оценить его точность.
- Ранговая корреляция Спирмена используется для отражения монотонной связи, включая нелинейные монотонные зависимости.
- Визуализация данных, учет выбросов, и множественные проверки являются важными практическими приемами при интерпретации корреляции.
- Корреляционный анализ может быть применен для изучения взаимосвязи между различными переменными, включая анализ множественных переменных и учет влияния третьей переменной.

Примеры на Python

1) РАСЧЁТ PEARSON И SPEARMAN + SCATTER PLOT С ЛИНИЕЙ РЕГРЕССИИ

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import pearsonr, spearmanr, linregress

# Пример данных
x = np.array([1,2,3,4,5,6,7,8,9,10])
y = np.array([2.3,2.9,3.8,5.1,5.0,6.2,7.1,7.9,9.2,9.7])

# Коэффициенты
r, pval_r = pearsonr(x, y)
rho, pval_rho = spearmanr(x, y)
print(f"Pearson r = {r:.3f}, p = {pval_r:.3e}")
print(f"Spearman rho = {rho:.3f}, p = {pval_rho:.3e}")

# График + линия наилучшей линейной аппроксимации
slope, intercept, r_val, p_val, stderr = linregress(x, y)
plt.scatter(x, y)
plt.plot(x, intercept + slope*x, linestyle='--') # линия регрессии
plt.xlabel("X")
plt.ylabel("Y")
plt.title(f"Scatter: r={r:.3f}, rho={rho:.3f}")
plt.show()
```

2) ТЕСТ ЗНАЧИМОСТИ И ДОВЕРИТЕЛЬНЫЙ ИНТЕРВАЛ ДЛЯ R (FISHER Z)

```
import numpy as np
from scipy.stats import norm

def fisher_ci(r, n, alpha=0.05):
    z = np.arctanh(r)
    se = 1/np.sqrt(n-3)
    z_crit = norm.ppf(1-alpha/2)
    lo = np.tanh(z - z_crit*se)
    hi = np.tanh(z + z_crit*se)
    return lo, hi

lo, hi = fisher_ci(r, len(x))
print(f"95% CI for r (Fisher): [{lo:.3f}, {hi:.3f}"])
```

3) ПРИМЕР ВЛИЯНИЯ ВЫБРОСА

```
import numpy as np
from scipy.stats import pearsonr

x2 = x.copy()
y2 = y.copy()
# добавим один выброс в конец
x2 = np.append(x2, 100)
y2 = np.append(y2, -50)

print("Без выброса:", pearsonr(x, y))
print("С выбросом:", pearsonr(x2, y2))
```

Урок 3. Корреляция Пирсона

В этом уроке мы сфокусируемся на классическом измерителе линейной связи — коэффициенте корреляции Пирсона. Разберём, откуда берётся формула, в каких случаях её можно применять, как сделать статистический вывод о значимости корреляции и как интерпретировать результаты на примере реальных данных. Также покажем практические вычисления p -value и доверительного интервала через преобразование Фишера.

- Корреляция Пирсона - классический метод оценки линейной связи между случайными величинами.
- Коэффициент корреляции Пирсона показывает, насколько отклонения одной переменной совпадают с отклонениями другой.
- Проверка значимости корреляции и построение доверительного интервала через преобразование Фишера.
- Важно провести разведывательный анализ данных, построить диаграмму рассеяния и проверить наличие линейности и выбросов.
- Корреляция не означает причинность, для причинных выводов необходимо проводить эксперименты или использовать специальные методы анализа.

Примеры на Python

1) БЫСТРЫЙ РАСЧЁТ R, T, P-VALUE И FISHER CI

```
import numpy as np
from scipy import stats

x = np.array([3.63,3.02,3.82,3.42,3.59,2.87,3.03,3.46,3.36,3.30])
y = np.array([53.1,49.7,48.4,54.2,54.9,43.7,47.2,45.2,54.4,50.4])
n = len(x)

# Pearson r и p-value (scipy)
r, p_two = stats.pearsonr(x, y)
print(f"Pearson r = {r:.3f}, two-sided p = {p_two:.3f}")

# t-статистика (вручную, проверка)
t_stat = r / np.sqrt((1 - r**2) / (n - 2))
print(f"t = {t_stat:.3f}, df = {n-2}")

# Fisher z CI
z = np.arctanh(r)
se_z = 1/np.sqrt(n-3)
z_crit = stats.norm.ppf(0.975)
lo = np.tanh(z - z_crit * se_z)
hi = np.tanh(z + z_crit * se_z)
print(f"95% Fisher CI for rho: [{lo:.3f}, {hi:.3f}]")
```

2) ВИЗУАЛИЗАЦИЯ И ПРОВЕРКА ВЫБРОСОВ

```
import matplotlib.pyplot as plt
from scipy import stats

slope, intercept, r_val, p_val, stderr = stats.linregress(x, y)

plt.figure(figsize=(6,4))
plt.scatter(x, y)
plt.plot(x, intercept + slope*x, linestyle='--', label=f"OLS line (r={r:.3f})")
plt.xlabel("Mass (kg)")
plt.ylabel("Height (cm)")
plt.title("Mass vs Height (newborns)")
plt.legend()
plt.show()
```

Урок 4. Корреляция Спирмена

В этом уроке мы подробно разберём ранговую корреляцию Спирмена — что она измеряет, зачем нужна, как считается вручную и в Python, как интерпретировать результаты и какие есть подводные камни — связи рангов, повторяющиеся значения, точность p-value.

- Корреляция Спирмена измеряет взаимосвязь между двумя переменными в ранговой шкале.
- Используется для монотонных (ранговых) связей, отличается от корреляции Пирсона.
- Свойства коэффициента корреляции Спирмена: $r_s \in [-1, 1]$, $r_s = 1$ для идеальной прямой, $r_s = -1$ для идеальной обратной связи.
- При наличии повторяющихся значений (ties) формула корректируется или применяются поправки.
- Для малых выборок и связей рангов рекомендуется использовать точные тесты или перестановочные тесты.
- Метод Спирмена более устойчив к выбросам, чем корреляция Пирсона.
- Корреляция - мера ассоциативности, не указывает на причинно-следственные связи.

Примеры на Python

а) БЫСТРЫЙ РАСЧЁТ SPEARMAN (SCIPY)

```
import numpy as np
from scipy import stats

x = np.array([106,100,86,101,99,103,97,112,113,110])
y = np.array([7,27,2,50,28,29,20,6,12,17])

rho, pval = stats.spearmanr(x, y)
print(f"Spearman rho = {rho:.3f}, p-value = {pval:.3f}")
```

`scipy.stats.spearmanr` автоматически обрабатывает связи рангов (ties) и возвращает p-value (точное или приближённое в зависимости от n и связей).

в) РУЧНОЙ ПОДСЧЁТ ЧЕРЕЗ РАНГИ И ФОРМУЛУ

```
import numpy as np
from scipy.stats import rankdata

Rx = rankdata(x, method='average') # ранги x (средние ранги при ties)
Ry = rankdata(y, method='average')
d = Rx - Ry
rs_manual = 1 - 6*np.sum(d**2)/(len(x)*(len(x)**2 - 1))
```

```
print("Rs manual:", round(rs_manual, 3))
# Также можно вычислить корреляцию Пирсона между рангами:
rs_via_pearson = np.corrcoef(Rx, Ry)[0,1]
print("Rs via pearson on ranks:", round(rs_via_pearson, 3))
```

С) ВИЗУАЛИЗАЦИЯ РАНГОВ И ИСХОДНЫХ ДАННЫХ

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))
plt.subplot(1,2,1)
plt.scatter(x, y)
plt.title("Исходные данные (IQ vs TV hours)")
plt.xlabel("IQ")
plt.ylabel("Hours")

plt.subplot(1,2,2)
plt.scatter(Rx, Ry)
plt.title("Ранги (R_x vs R_y)")
plt.xlabel("Rank(X)")
plt.ylabel("Rank(Y)")
plt.tight_layout()
plt.show()
```

Д) ПЕРЕСТАНОВОЧНЫЙ ТЕСТ (ЕСЛИ ХОЧЕТСЯ ТОЧНОГО P-VALUE ДЛЯ МАЛЫХ N)

```
from scipy import stats
import numpy as np

def perm_test_spearman(x, y, B=10000, seed=1):
    rng = np.random.default_rng(seed)
    obs = stats.spearmanr(x, y).correlation
    count = 0
    for _ in range(B):
        y_perm = rng.permutation(y)
        if abs(stats.spearmanr(x, y_perm).correlation) >= abs(obs):
            count += 1
    return (count+1)/(B+1)

p_perm = perm_test_spearman(x, y, B=5000)
print("Permutation p-value (two-sided):", p_perm)
```

Перестановочный тест даёт точный эмпирический p-value и полезен при малом n или при сомнении в применимости асимптотических приближений.

Урок 5. Регрессионный анализ

В этом уроке мы переходим от оценки наличия связи к её использованию: регрессии — методу, который позволяет предсказывать значение одной переменной по значениям другой(их). Разберём простейшую и самую важную модель — линейную регрессию (обыкновенные наименьшие квадраты, OLS) — от интуиции и формул до интерпретации, диагностики и практической реализации в Python.

- Регрессионный анализ позволяет предсказывать значение одной переменной по значениям другой.
- Линейная регрессия - простейшая и самая важная модель, которая позволяет предсказывать значение одной переменной по значениям другой.
- Ключевые допущения: линейность, независимость ошибок, гомоскедастичность, нормальность ошибок, отсутствие мультиколлинеарности.
- OLS минимизирует сумму квадратов остатков, а суммарно (матрицы) $\hat{\beta} = (X^T X)^{-1} X^T y$.

- Интервалы и предсказания: доверительные и предсказательные интервалы, коэффициент детерминации R^2 .
- Практические рекомендации: проверка допущений регрессионного анализа, диагностика остатков и проверка статистических свойств модели.
- Примеры на Python: использование библиотеки statsmodels и NumPy для анализа данных.

Примеры на Python

А) STATSMODELS – ПОЛНЫЙ ВЫВОД, CI, ПРЕДСКАЗАНИЯ

```
import numpy as np
import statsmodels.api as sm

# данные (масса -> рост)
x = np.array([3.63,3.02,3.82,3.42,3.59,2.87,3.03,3.46,3.36,3.30])
y = np.array([53.1,49.7,48.4,54.2,54.9,43.7,47.2,45.2,54.4,50.4])

X = sm.add_constant(x)
model = sm.OLS(y, X).fit()
print(model.summary())

# предсказание и интервалы для x0
x0 = np.array([1, 3.5]) # [const, x0]
pred = model.get_prediction(x0)
print(pred.summary_frame(alpha=0.05)) # mean, mean_ci_lower/upper, obs_ci_lower/upper

statsmodels сразу даёт коэффициенты, se, t, p-values,  $R^2$ , скорректированный  $R^2$ , F-статистику, а также удобные методы для CI и prediction intervals.
```

В) NUMPY / ВРУЧНУЮ

```
import numpy as np
from scipy.stats import t

x = np.array([3.63,3.02,3.82,3.42,3.59,2.87,3.03,3.46,3.36,3.30])
y = np.array([53.1,49.7,48.4,54.2,54.9,43.7,47.2,45.2,54.4,50.4])

n = len(x)
xbar, ybar = x.mean(), y.mean()
Sxx = np.sum((x - xbar)**2)
Sxy = np.sum((x - xbar)*(y - ybar))

b1 = Sxy / Sxx
b0 = ybar - b1*xbar
y_pred = b0 + b1*x
resid = y - y_pred
SSE = np.sum(resid**2)
s2 = SSE/(n-2)
se_b1 = np.sqrt(s2 / Sxx)

# 95% CI for slope
tcrit = t.ppf(0.975, n-2)
ci_b1 = (b1 - tcrit*se_b1, b1 + tcrit*se_b1)
print(b1, ci_b1)
```

Урок 6. Парная линейная регрессия

В предыдущих уроках мы знакомились с тем, как выявлять взаимосвязи между признаками (корреляцию). Теперь посмотрим, как использовать эту связь для предсказаний: в простейшем случае — когда одна переменная предсказывает другую с помощью линейной модели (парная линейная регрессия).

- Парная линейная регрессия используется для предсказания одной переменной через другую.
- Задача - по данным оценить a и b так, чтобы предсказания $Y_i = a + bX_i$ максимально согласовывались с наблюдаемыми Y_i .
- Метод наименьших квадратов (OLS) основан на идее минимизации суммы квадратов отклонений фактических значений зависимой переменной Y_i от предсказанных моделью значений $\hat{Y}_i$.
- Стандартные ошибки и t -статистика используются для проверки нулевой гипотезы $H_0: b=0$.
- Коэффициент детерминации R^2 показывает долю объяснённой дисперсии зависимой переменной.
- Интервалы прогноза и предсказания строятся для нового значения x_0 .
- Практические рекомендации: при малой выборке не полагайтесь только на p -value, используйте CI и предсказательные интервалы; если остатковая дисперсия растёт/убывает по X - есть гетероскедастичность → используйте робастные стандартные ошибки или WLS; для стабильных предсказаний на новых данных имеет смысл оценивать модель на hold-out и сравнивать с регуляризованными методами.

Пример на Python

А) ПОЛНАЯ СВОДКА ЧЕРЕЗ STATSMODELS

```
import numpy as np
import statsmodels.api as sm

x = np.array([3.63, 3.02, 3.82, 3.42, 3.59, 2.87, 3.03, 3.46, 3.36, 3.30])
y = np.array([53.1, 49.7, 48.4, 54.2, 54.9, 43.7, 47.2, 45.2, 54.4, 50.4])
X = sm.add_constant(x)
model = sm.OLS(y, X).fit()
print(model.summary())

# Предсказание с интервалами для  $x_0=3.5$ 
x0 = np.array([1, 3.5])
pred = model.get_prediction(x0)
print(pred.summary_frame(alpha=0.05))
```

Б) ВРУЧНУЮ НА NUMPY

```
import numpy as np
from scipy import stats

# данные x,y как выше
n = len(x)
xbar, ybar = x.mean(), y.mean()
Sxx = np.sum((x - xbar)**2)
Sxy = np.sum((x - xbar)*(y - ybar))

b1 = Sxy / Sxx
b0 = ybar - b1*xbar
yhat = b0 + b1*x
resid = y - yhat
SSE = np.sum(resid**2)
SST = np.sum((y - ybar)**2)
R2 = 1 - SSE/SST
```

```

s2 = SSE/(n-2)
se_b1 = np.sqrt(s2 / Sxx)
se_b0 = np.sqrt(s2*(1/n + xbar**2/Sxx))
t_b1 = b1 / se_b1
F = (R2/(1-R2))*(n-2)
# прогноз и предсказательный интервал для x0
x0 = 3.5
y0 = b0 + b1*x0
se_pred = np.sqrt(s2*(1 + 1/n + (x0 - xbar)**2 / Sxx))
tcrit = stats.t.ppf(0.975, n-2)
pi_lo, pi_hi = y0 - tcrit*se_pred, y0 + tcrit*se_pred

```

Урок 7. Множественная линейная регрессия

В этом уроке мы расширим идею парной регрессии на случай нескольких объясняющих переменных. Цель — научиться строить линейную модель

$$Y_i = a + b_1 X_{i1} + b_2 X_{i2} + \dots + b_m X_{im} + \varepsilon_i,$$

правильно её интерпретировать, проверять значимость и диагностировать проблемы, типичные для множественных моделей.

- Множественная линейная регрессия расширяет парную регрессию на случай нескольких объясняющих переменных.
- Цель - научиться строить линейную модель $Y_i = a + b_1 X_{i1} + b_2 X_{i2} + \dots + b_m X_{im} + \varepsilon_i$, правильно её интерпретировать, проверять значимость и диагностировать проблемы.
- Матричная запись и оценивание: $Y = X\beta + \varepsilon$, $\hat{\beta} = (X^T X)^{-1} X^T Y$.
- Стандартизация: $t_X = X - \bar{X}/s_X$, $t_Y = Y - \bar{Y}/s_Y$.
- Мультиколлинеарность: $VIF_j = 1/(1-R_j^2)$.
- Отбор переменных: полный или ручной отбор, метод вперёд, метод назад, шаговый метод.
- Оценка качества модели: коэффициент детерминации R^2 и F-тест общей значимости модели.
- Практические рекомендации: использовать $\bar{R}^2$ при сравнении моделей с разным числом предикторов, проверять F-тест для общей значимости, не ориентироваться только на R^2 , всегда проверять качество предсказаний на новых данных.

Пример на Python

Ниже — небольшой демонстрационный пример с искусственными данными (чтобы показать диагностику VIF и стандартизацию). Используем 2 предиктора X_1, X_2 и отклик Y . Для наглядности Y задан как точная линейная комбинация (чтобы коэффициенты были тривиальны).
Данные $n=10$:

```

# X1, X2, Y (пример)
X1 = [1,2,3,4,5,6,7,8,9,10]
X2 = [2,1,4,3,5,7,6,8,9,10]
Y = 3 + 2*X1 + 0.5*X2    # точная модель без шума

```

STATSMODELS + ВЫЧИСЛЕНИЕ VIF

```

import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# данные
df = pd.DataFrame({

```

```

"X1": [1,2,3,4,5,6,7,8,9,10],
"X2": [2,1,4,3,5,7,6,8,9,10],
})
df["Y"] = 3 + 2*df["X1"] + 0.5*df["X2"]

# OLS
X = sm.add_constant(df[["X1", "X2"]])
y = df["Y"]
model = sm.OLS(y, X).fit()
print(model.summary())

# стандартизация
X_std = (df[["X1", "X2"]] - df[["X1", "X2"]].mean()) / df[["X1", "X2"]].std(ddof=0)
y_std = (df["Y"] - df["Y"].mean()) / df["Y"].std(ddof=0)
model_std = sm.OLS(y_std, sm.add_constant(X_std)).fit()
print("Standardized coefficients (approx):", model_std.params)

# VIF
vif_data = pd.DataFrame()
vif_data["feature"] = ["const", "X1", "X2"]
vif_data["VIF"] = [np.nan] + [
    variance_inflation_factor(X.values, i+1) for i in range(2)
]

print(vif_data)

```

Примечание: `variance_inflation_factor` ожидает матрицу с константой в первом столбце; VIF для const обычно не интерпретируют.

Урок 8. Градиентный спуск

В предыдущих уроках мы решали задачу минимизации квадрата ошибок для линейной регрессии в явном виде (формулы OLS). Однако на практике часто приходится минимизировать более общие функционалы качества — суммы модулей ошибок, регуляризованные функции, сложные нейросетевые лоссы и т.д. Для этих задач стандартный инструмент — градиентный спуск.

- Градиентный спуск - метод нахождения локальных минимумов или максимумов функции путем итеративного движения в направлении, противоположном градиенту.
- Градиент функции - вектор, каждая компонента которого равна частной производной функции по соответствующему параметру.
- Направление градиента указывает, куда функция растёт быстрее всего, и используется для минимизации функционала.
- Инициализация: выбор начального приближения $\beta^{(0)}$.
- Вычисление градиента: вычисление $\nabla_{\beta} Q(\beta^{(t)})$ на каждой итерации t .
- Обновление параметров: $\beta^{(t+1)} = \beta^{(t)} - \eta \nabla_{\beta} Q(\beta^{(t)})$.
- Критерии остановки: норма градиента, изменение функционала между итерациями, длина шага.
- Выбор шага обучения η и критерия остановки ζ важен для сходимости.
- Модификации градиентного спуска: стохастический градиентный спуск, мини-батч градиентный спуск, моментный градиентный спуск, Nesterov ускоренный градиентный спуск.

Примеры на Python

1) ГРАДИЕНТНЫЙ СПУСК ДЛЯ КВАДРАТИЧНОЙ ФУНКЦИИ

```
import numpy as np
```



```
def gd_quadratic(eta=0.1, iters=10):
    w = 0.0
    hist = [w]
    for t in range(iters):
        grad = 2*(w - 3)      # производная f'(w) = 2(w-3)
        w = w - eta * grad
        hist.append(w)
    return np.array(hist)

print(gd_quadratic(eta=0.1, iters=3)) # ожидаем [0.0, 0.6, 1.08, 1.464]
```

2) ГРАДИЕНТНЫЙ СПУСК ДЛЯ ПРОСТОЙ ЛИНЕЙНОЙ РЕГРЕССИИ (MSE), БАТЧЕВЫЙ ВАРИАНТ

```
import numpy as np

# небольшие синтетические данные
x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([3.0, 5.0, 7.0, 9.0]) #  $y \approx 1 + 2 \cdot x$ 

# инициализация
w = 0.0
b = 0.0
eta = 0.01
n_iter = 1000
n = len(x)

for epoch in range(n_iter):
    # предсказание
    y_hat = w * x + b
    # градиенты для MSE (с фактором 2/n)
    grad_w = (-2.0/n) * np.sum(x * (y - y_hat))
    grad_b = (-2.0/n) * np.sum(y - y_hat)
    # обновление
    w -= eta * grad_w
    b -= eta * grad_b

print("w, b:", round(w,3), round(b,3)) # ожидаем примерно  $w \approx 2$ ,  $b \approx 1$ 
```

Для практических задач вместо ручного батчевого GD часто используют `sklearn.statsmodels` (для быстрого OLS) или оптимизаторы из `deep-learning` фреймворков (Adam, RMSprop и т.д.) для сложных/невыпуклых целей.

Урок 10. Заключение

В рамках этого модуля мы изучали задачи определения взаимосвязей между величинами и предсказания неизвестных значений на основе имеющихся данных. Мы познакомились с коэффициентами корреляции Пирсона и Спирмена, которые позволяют выявлять зависимость между двумя величинами: Пирсон применим для нормально распределённых данных, а Спирмен — для произвольных распределений и более устойчив к выбросам.

Далее мы рассмотрели линейную регрессию, с помощью которой можно прогнозировать значение целевой переменной как на основе одной известной величины (парная регрессия), так и нескольких факторов одновременно (множественная регрессия). Мы научились оценивать вклад каждой переменной через весовые коэффициенты и анализировать качество модели с помощью R^2 , доверительных интервалов и F-тестов. В завершение модуля мы кратко познакомились с градиентным спуском как универсальным методом оптимизации, используемым во всех типах регрессионных моделей, и обсудили его основные параметры и практические аспекты применения.

За время курса «Специальные разделы математической статистики» мы:

- освежили основы теории вероятностей и научились вычислять вероятности событий и характеристики выборок;
- изучили типы распределений данных, их особенности и методы визуализации;
- разобрали постановку гипотез, ошибки первого и второго рода, а также применение центральной предельной теоремы к выборкам;
- освоили основные статистические тесты для проверки значимости получаемых результатов;
- познакомились с различными методами анализа данных — корреляционным, дисперсионным и регрессионным — для изучения взаимосвязей и прогнозирования неизвестных величин.

Полученные знания создают прочную базу, которая пригодится в аналитике данных, инженерии данных и задачах машинного обучения.

Желаем успехов в дальнейшей практике и исследованиях!